

autoresearch-rsi-onefile.py: A Technical HOWTO

Recursive self-improvement at the exploration layer on a local multi-tier inference cluster

ThereminQ / thereminq-autoresearch. Technical documentation, revision of 2026-09-27 (hardening release: contract forge, container discovery, grounded verification, cross-round iteration, compounding RSI, token accounting). The paper-aligned core of 2026-09-19 is unchanged; §17 describes the layer built around it and §18 lists every change.

Abstract

This document is an operational and technical description of autoresearch-rsi-onefile.py, a single-file orchestration pipeline. The pipeline turns a natural-language prompt, a prompt file, or a Git repository into a partitioned set of agent assignments. A team of language-model agents on a local inference cluster then executes those assignments over several recursive rounds. The central mechanism follows the Dream-RSI framework of Zheng et al. [1]:

- An exploration policy, represented as executable code, acts in **decision rounds**. In each round it selects a batch of at most W legal continuations of the discovery tree, and it stops by submitting no further batch.
- A **fixed evaluator** scores each attempt once, at creation. This includes unit tests generated and executed for the attempt’s deliverables.
- Each online rollout is recorded as a discovery tree. The accumulated trees serve as a replay simulator in which policy versions are evaluated at zero agent-call cost (“dreaming”), using the replay objective of [1, Eq. 1].
- A policy-development agent derives a chain of revisions, each from its predecessor’s replay trajectories and scores. The argmax over all versions is redeployed in the next round.

Since the paper-aligned release, a **verification and hardening layer** has been built around this loop without changing its formalism. The layer:

- turns even a short prose prompt into a pinned contract with a planner-chosen shared API, a run command, a negative control, ablations and validated known answers;
- discovers the abilities of the container it runs in (installed Python modules, the real APIs of the libraries in use, compute devices) instead of reading static lists;
- runs the integrated project after every round and checks, with a noise-aware negative control and ablations, that its metrics measure something;
- detects and penalizes shortcuts: hard-coded metrics, invented library calls, wrong argument counts, hollow known-answer tests;
- routes project-level failures to the agent whose file has to change;
- iterates code across rounds instead of rewriting it, prunes dead lines of work, and carries dreamed policies across runs;
- accounts for every token and reports throughput per category.

The layer was hardened deliberately against fast, error-prone models, whose failures exposed each gap. §17 describes it; §18 lists the changes with the failure that motivated each.

We describe the system architecture, all command-line options and environment variables, the per-phase workflow, the evaluator and replay objective, and the policy programming interface with its sandbox. We also cover inter-agent communication, the on-disk artefact layout, and standard operating procedures. We conclude with a precise account of how this implementation

maps onto the formalism of [1], and of the few adaptations that its multi-assignment setting requires.

Contents

1. [Introduction and scope](#)
 2. [Background: exploration-layer RSI](#)
 3. [System architecture](#)
 4. [Installation and cluster deployment](#)
 5. [Command-line interface](#)
 6. [Configuration by environment variables](#)
 7. [Workflow: phase-by-phase](#)
 8. [Evaluator and replay objective](#)
 9. [Writing exploration policies](#)
 10. [Inter-agent communication](#)
 11. [Run directory layout](#)
 12. [Standard operating procedures](#)
 13. [Correspondence with Dream-RSI](#)
 14. [Security model](#)
 15. [Diagnostics and troubleshooting](#)
 16. [Known limitations](#)
 17. [Verification and hardening layer](#)
 18. [Change log since the paper-aligned release](#)
 19. [References](#)
 20. [Appendices](#)
-

1. Introduction and scope

1.1 Problem setting

Long-horizon, agent-driven discovery spends most of its compute on proposal-evaluation cycles. How those cycles are allocated is decided by the *exploration strategy*: which branches to open, which to refine, what to run in parallel, and when to stop. Zheng et al. [1] identify two obstacles to improving that strategy:

- **Delayed feedback.** A strategy’s quality is only observable after a long rollout.
- **A large meta-search space.** Many candidate strategies may need to be tried.

Their proposal is to treat completed discovery histories as a replay simulator. Alternative strategies can then be scored against recorded outcomes without re-invoking the discovery agent or the evaluator. This is analogous to model-based reinforcement learning and world models [2-6].

1.2 What this script is

`autoresearch-rsi-onefile.py` (a single Python module of about 10,200 lines) implements that loop for a multi-agent research-and-build pipeline running on self-hosted, OpenAI-compatible inference servers (e.g. `llama-server` from `llama.cpp` [13]). It has six properties.

1. **The unit of exploration is an agent assignment.** A planning model partitions the input into mutually exclusive assignments $t_{01}, \dots, t_N$ ($3 \leq N \leq 20$). Each assignment is an independent discovery problem. All assignments share the root of one discovery tree per round.
2. **One online attempt is one agent call plus its evaluation.** The attempt writes real files into an agent-owned directory. The fixed evaluator scores it, and it becomes a tree node carrying its realized outcome.

3. **Exploration follows the decision-round interface of [1, §3].** Legal continuations are the root (a new branch) and the current leaves. A batch holds at most W of them. Rollouts are capped at K_1 decision rounds online and K_2 in replay.
4. **Only the exploration policy is improved.** The models, prompts, evaluator and execution interfaces are fixed [1, §3].
5. **Policy improvement follows [1, §3].** M chained policy versions are replayed on the fixed history, scored by Eq. (1) with its cost and parallelism terms, and the argmax is deployed.
6. **Results are verified, not just well-formed.** Around the loop sits a verification layer (§17): pinned contracts, container discovery, grounded runs with a negative control and ablations, shortcut detection, routed findings and cross-round code iteration. It changes *what the evaluator measures*, never *how policies are improved*.

1.3 Status of the reference implementation

The Dream-RSI paper is available as arXiv:2609.14858 [1]. At the time of writing, the official repository zhengkid/Dream-RSI [14] contains the paper, the project assets and a citation file. Its release plan lists the full codebase, the discovered programs and the reproduction scripts as “being prepared”. Consequently:

- this script is an **independent implementation** of the formalism in §3 of [1];
- it is not a port of released code;
- it adopts the parts of the published prompts (Appendix B of [1]) that concern the policy interface: legal-action lists, the maximum batch size, prefix-only decisions, and per-round execution traces as between-version feedback;
- §13 lists the correspondence element by element, together with the adaptations that remain.

2. Background: exploration-layer RSI

We briefly restate the formalism of [1, §3] in the notation used throughout.

Discovery tree. A rollout grows a rooted tree $\mathcal{T}$. Each non-root node v records a single attempt: its parent, the resulting workspace/artefacts, diagnostics, and a score s_v (larger is better). The atomic action $\text{Continue}(v)$ resumes the workspace of v and produces one new evaluated child. Continuing from the root opens a new branch.

Policy and batches. With W parallel workers, a policy selects at each decision round a batch C of at most W eligible nodes. The empty batch terminates the rollout.

Online rollout. At outer iteration t , policy π_t is deployed and produces $\mathcal{T}_t$. The history grows to $\mathcal{H}_t = (\mathcal{T}_1, \dots, \mathcal{T}_t)$.

Replay. A candidate policy is replayed from the root of each recorded $\mathcal{T}_i$. Selecting v deterministically reveals the recorded child of v , if any. The policy sees only the revealed prefix.

Replay objective [1, Eq. 1]:

$$V_i^m = \max_{v \in \hat{\mathcal{T}}_i^m} s_v - \beta_1 N_i^m + \beta_2 \frac{N_i^m}{\max\{1, k_i^{m,*}\}},$$

where N_i^m is the number of revealed non-root nodes and $k_i^{m,*}$ the number of decision rounds.

Selection. The policy-development agent writes revisions $\pi_t^1, \dots, \pi_t^{M-1}$. The next policy is $\pi_{t+1} = \arg \max_m \frac{1}{t} \sum_i V_i^m$. Revision π_t^{m+1} is derived from π_t^m , using its replay trajectories and scores together with the feedback of earlier revisions. Because $\pi_t^0 = \pi_t$ is itself a candidate, the selected policy is never worse than the deployed one in average replay score on $\mathcal{H}_t$.

Section 13 details how each of these elements is realized, and where the script departs from the formalism.

3. System architecture

3.1 Inference tiers

The script addresses two tiers of OpenAI-compatible chat-completion endpoints. There is deliberately **no third “stitcher” tier**: no model ever merges the outputs of multiple agents. Consolidation is mechanical (a filesystem walk plus content hashing, §7.7).

Tier	Endpoint variable(s)	Default	Used for
Apex (plan- ning)	OPENAI_API_BASE, LLM_MODEL; DISTILLER_URL, DISTILLER_MODEL	http://localhost:9933/v1	Phase 1 generation, Phase 2 distillation, contract and interface synthesis, Phase 3 partitioning, policy development (dreaming), skeptic review, Phase 6 distillation
Workers (agents)	WORKER_ENDPOINTS, WORKER_MODEL	http://localhost:9931/v1, http://localhost:9932/v1 (at least MIN_WORKER_ ENDPOINTS = 2)	Every agent assignment (online and rebuttal rounds), Phase 0 repository map-reduce, evaluator test generation, the final write-up refresh

The apex tier is assumed to be single-slot (APEX_SERVER_NP=1) and is therefore used strictly serially. Each worker endpoint contributes WORKER_PARALLEL_SLOTS concurrent slots to a shared slot queue. The number of parallel workers W , i.e. the maximum batch size of one decision round, defaults to

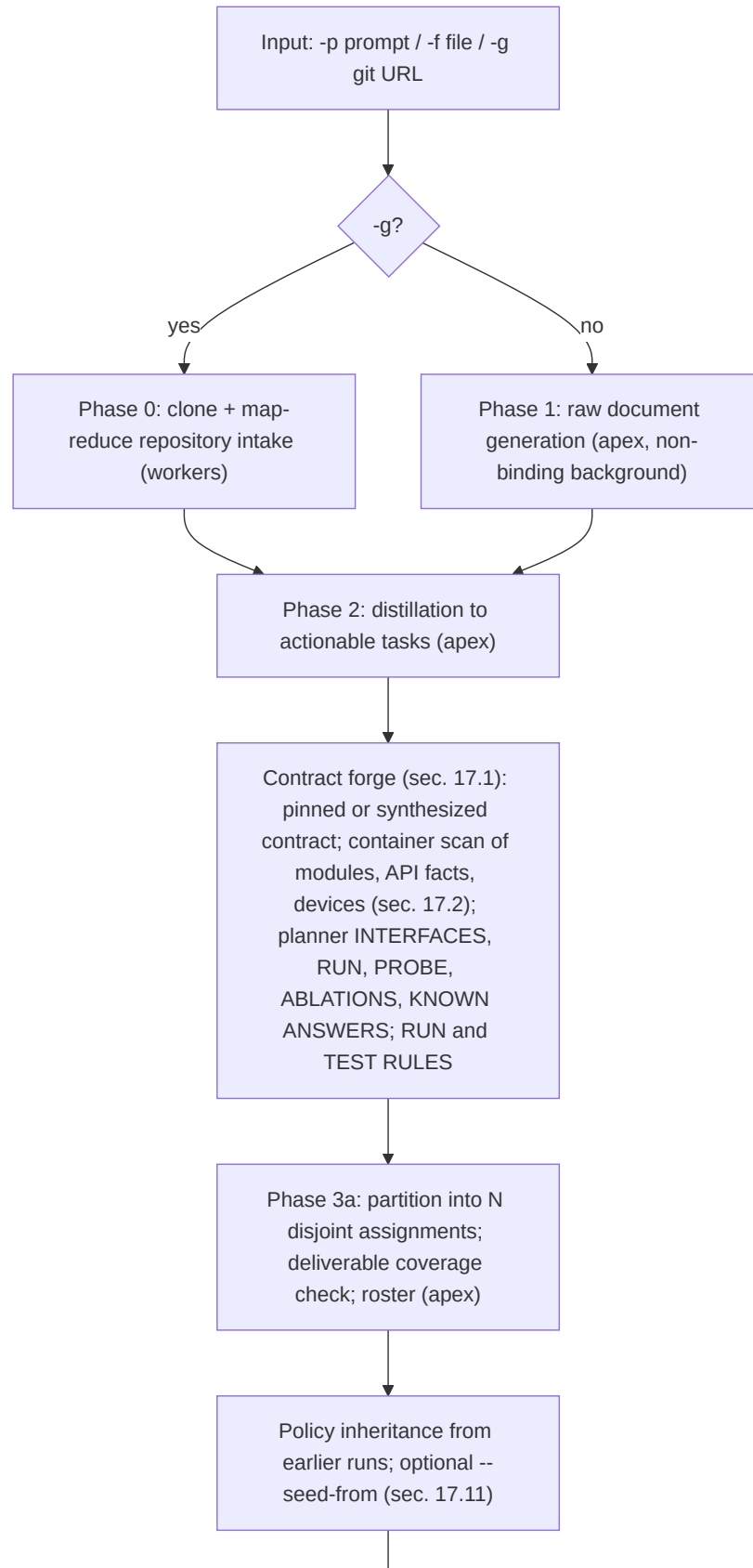
$$W = |\text{WORKER_ENDPOINTS}| \times \text{WORKER_PARALLEL_SLOTS}$$

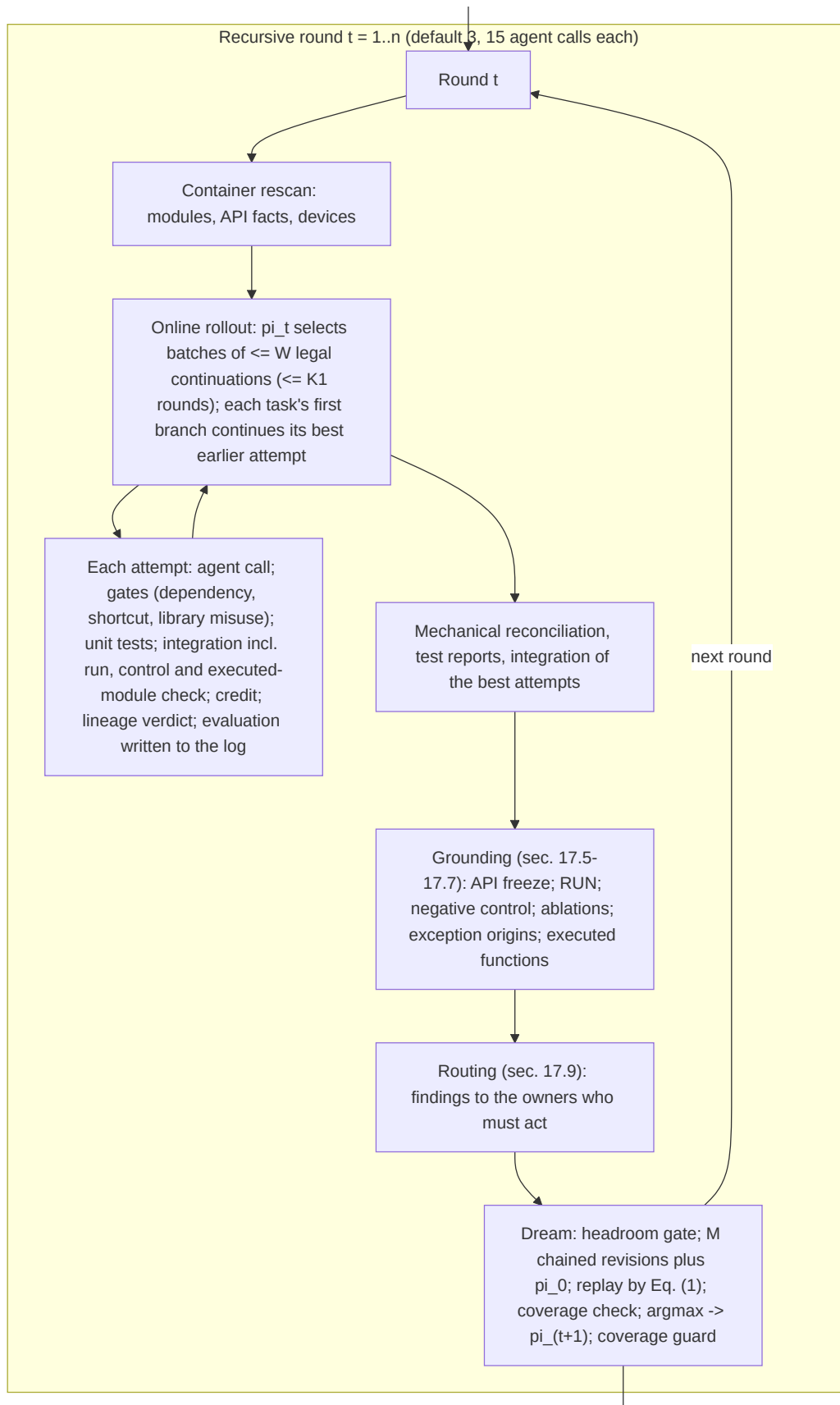
and can be overridden with MAX_PARALLELISM. The same W applies in replay.

Note on port numbers. The apex defaults to port 9933 and the workers to ports 9931 and 9932. At least MIN_WORKER_ENDPOINTS (default 2) distinct worker endpoints must be configured *and* answer the smoke test, otherwise the run refuses to start. Listing the apex among the workers is allowed but warned about: its single slot would queue behind agent calls. Environment variables override these defaults; if a container still exports an older OPENAI_API_BASE or WORKER_ENDPOINTS, update it (§4.2).

Note on gateways. Any OpenAI-compatible endpoint works: llama-server locally, or a gateway such as the repository’s openai2gemini.py, which the hardening runs used to drive fast, error-prone hosted models. The script adapts to endpoint quirks at run time: sampling penalties that an endpoint rejects are dropped and remembered per endpoint, and transient upstream errors (HTTP 429/5xx, “high demand”, dropped connections) are retried with backoff (§6.11, §17.13).

3.2 Phase structure





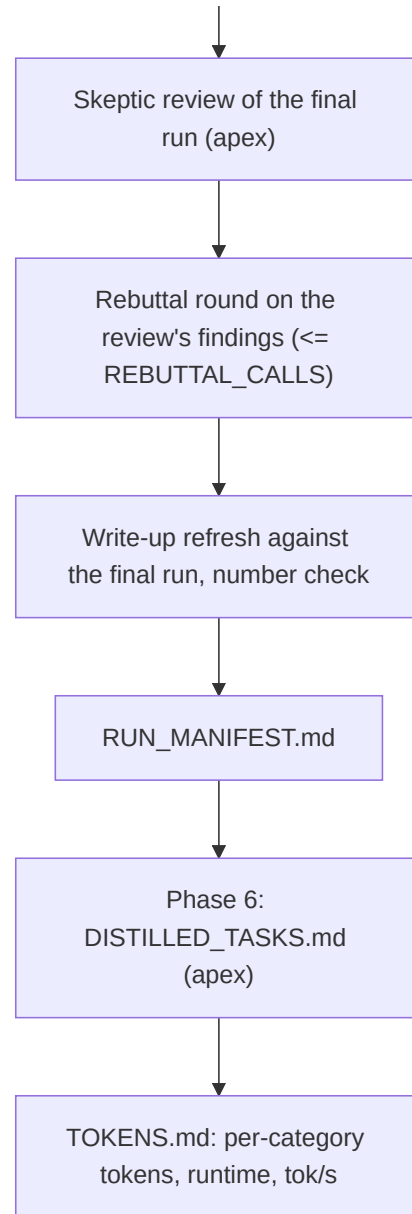


Figure 1. Phase structure (the Mermaid source is in the Markdown version).

3.3 Separation of concerns

The design rests on six invariants. The rest of the code is organized to preserve them.

1. **Identical policy surface.** The same policy source runs unchanged against LiveExplorer (real agent calls) and ReplayExplorer (recorded nodes, no calls). Both enforce the same decision-round and legality rules. This identity is what makes dreaming meaningful.
2. **A single, final score per node.** The evaluator is applied once, at creation, and its score is never rewritten. The online policy decides on exactly the value that replay later reveals, so replay is exact on recorded branches.
3. **Budget in one currency.** One unit equals one discovery-agent call, and retries are charged. The currency is identical online and in replay. Evaluator calls (test generation) are not discovery-agent calls and are not charged, as in [1], where the evaluator is outside the discovery budget.
4. **Ownership.** Every agent writes only into its own directory. Cross-agent paths are quarantined as violations (§7.5).
5. **The container defines the workload's abilities.** Importable modules, the real APIs of the libraries in use, and compute devices are *discovered*, never configured. The container's owner can install more at any time; the next scan picks it up (§17.2).

6. **Numbers must be earned.** A result counts only if the project actually runs, its metrics respond *by measurement* to a negative control, and the write-up quotes only what the run printed. Shortcuts are detected statically and penalized (§17.3–17.6).

4. Installation and cluster deployment

4.1 Host requirements

Requirement	Reason
Linux (POSIX)	resource.setrlimit, os.killpg, ulimit and process groups are used by the policy sandbox and the test runner.
Python ≥ 3.9	ThreadPoolExecutor.shutdown(cancel_futures=True), modern typing.
openai (Python SDK), requests	Chat completions (streaming) and Phase 5 raw HTTP calls.
git on PATH	Phase 0 (-g).
gcc, g++	Phase 5 C/C++ tests.
pytest (host or venv)	Phase 5 Python tests. The script creates tests/.venv with --system-site-packages and installs pytest from binary wheels if it is missing.
bash	Phase 5 shell tests and the ulimit wrapper.

```
python3 -m pip install openai requests pytest
```

4.2 Server context alignment

The script’s token budgets must mirror the `-c` and `-np` flags of the inference servers. For llama-server with a unified KV cache (`--kv-unified`), `-c N` is the **total** KV budget of the process, shared across the `-np` slots. The concurrency-safe per-request window is therefore $\lfloor N/n_p \rfloor$. The script computes

$$\begin{aligned} \text{APEX_CONTEXT_TOKENS} &= \max(4096, \lfloor \text{APEX_SERVER_CTX} / \text{APEX_SERVER_NP} \rfloor), \\ \text{WORKER_CONTEXT_TOKENS} &= \max(4096, \lfloor \text{WORKER_SERVER_CTX} / \text{WORKER_SERVER_NP} \rfloor). \end{aligned}$$

If the worker pool is heterogeneous in `-c`, set `WORKER_SERVER_CTX` to the **smallest** node’s value. Otherwise the larger nodes’ budgets will silently over-subscribe the smaller ones.

A minimal launch sketch (adapt model paths, ports and GPU layers to your hardware; building llama.cpp with its Vulkan backend avoids a vendor-specific GPU stack):

```
# Apex: single slot, 64k total context
llama-server -m apex.gguf --port 9933 -c 65536 -np 1 &

# Workers (at least two endpoints): two slots each, unified KV,
# 192k total context per process (96k per slot)
for p in 9931 9932; do
  llama-server -m worker.gguf --port $p -c 196608 -np 2 --kv-unified &
done
```

For hardening runs against a hosted model, run one gateway instance per port instead (e.g. openai2gemini.py on 9931, 9932 and 9933). The script needs no other change.

At start-up the script performs two checks:

- **verify_server_props** queries GET /props on every endpoint and reports each as ok, UNDERPROVISIONED, larger than budget or MISMATCH.
- **ping_tier** sends a four-token smoke completion. **Every** worker endpoint must pass, and at least `MIN_WORKER_ENDPOINTS` distinct worker endpoints must be configured, otherwise the run aborts before any work. If the apex fails, only apex-dependent phases are refused. A

misconfigured gateway (for example a model name the upstream does not know) therefore fails here, in seconds, rather than mid-run.

4.3 Budget alignment report

`describe_budget_alignment()` prints a [BUDGET] block at start-up: per-request windows, the agent input sub-budgets, the output ceiling, and the RSI settings. Read it before every new deployment. It prints two warnings that indicate misconfiguration:

- [!] Agent sub-budgets total ... exceeds the input window
- [!] WARNING: Calculated agent output budget collapsed to ... and hit the 1024 floor

The latter means that most agent deliverables will be truncated.

5. Command-line interface

```
autoresearch-rsi-onefile.py (-p PROMPT | -f FILE | -g GIT_URL | -r)
                           [--focus TEXT] [--git-path PATH]
                           [-d DIR] [-c CATEGORY]
                           [-n ROUNDS] [--budget CALLS]
                           [--no-dream | --dream-only] [--force-dream]
                           [--seed-from latest|RUN_DIR] [--integration-cmd CMD]
                           [--semantic-guidance] [--iterate]
```

Option	Type / default	Semantics
-p, --prompt	str	Direct prompt. Triggers Phase 1 (apex generation). Mutually exclusive with -f, -g.
-f, --file	path	Prompt read from a text file (non-ASCII stripped). Triggers Phase 1.
-g, --git	URL	Git repository as input. Triggers Phase 0 instead of Phase 1. The URL must match an https, http, ssh, git@ or git:// pattern and must not resolve to a private address unless GIT_ALLOW_PRIVATE_HOSTS=1.
--focus	str, ""	Analysis focus appended to the Phase 0 summarize/reduce prompts. Requires -g.
--git-path	str, ""	Restrict Phase 0 ingestion to one file or subdirectory of the repository. Requires -g.
-d, --dir	path, run_data	Base output directory.
-c, --category	str, projects	Category sub-folder. Runs are created as <dir>/<category>/run_<YYYYmdd_HHMMSS>_<hex6>/.
-r, --resume	flag	Bind to the most recently modified run_* directory in the category that contains at least one .md file, and continue from the furthest completed artefact. Cannot be combined with -p, -f, -g.
-n, --rounds	int, RSI_ROUNDS (3)	Total number of recursive rounds, $1 \leq n \leq \text{RSI_MAX_ROUNDS}$ (12). On resume, rounds already completed count towards n . To run one more round after three, pass -r -n 4. The rebuttal round (\$17.10) is extra and does not count.
--budget	int, 0 (auto)	Discovery-agent calls per round (the per-round resource cap). If the optional support-probe extension is enabled, this includes its reserve. 0 selects the default: a fixed ROUND_BUDGET (15), or the roster-scaled budget if ROUND_BUDGET_PER_TASK is set (\$7.4).
--no-dream	flag	Disable offline policy improvement. Every round redeploys the hand-written π_0 . This is the <i>Recursive Fixed Exploration</i> control of [1, \$4], at the same per-round budget.
--dream-only	flag	Run no agents. Dream over the existing tree pool of a resumed run and write the next policy file. Requires -r and at least one recorded round. Only the apex tier must be reachable. Contradicts --no-dream.
--force-dream	flag	Run the apex policy revisions even when the replay oracle bound shows less than DREAM_MIN_HEADROOM of headroom (\$7.9).
--seed-from	latest or run dir	Start round 1 from an earlier run's final project: every assignment whose deliverables all exist in that run's integration/latest/ is continued instead of rewritten (\$17.11).

Option	Type / default	Semantics
--integration-cmd	str	Command run inside every assembled project as an extra integration group (its "score" line is parsed). Without it, the RUN command of the contract is used for the per-attempt run check (§17.4).
--semantic-guidance	flag	Inject a fixed "directional guidance" section into every agent's communication digest. Off by default: it exists only to reproduce the negative ablation of [1, §5.1] (Fig. 5), in which prompt-level directional guidance underperformed unguided replay at equal budget.
--iterate	flag	Phase 6 refines an existing DISTILLED_TASKS.md instead of skipping it on resume.

Argument validation is strict. For example, --focus without -g, --dream-only without -r, or -n 13 are rejected by argparse before any network activity.

6. Configuration by environment variables

All tunables are read once at import time via `os.getenv`. Values are given with their coded defaults.

6.1 Endpoints and models

Variable	Default	Meaning
OPENAI_API_BASE	http://localhost:9933/v1	Apex base URL (Phases 1, 3a, contract and interface synthesis, dreaming, review).
OPENAI_API_KEY	sk-local	Apex API key.
LLM_MODEL	Qwen3.8-Flash-Next-UD-IQ4_XS	Apex model name.
DISTILLER_URL	http://localhost:9933/v1	Endpoint for Phase 2 and Phase 6 distillation.
DISTILLER_MODEL	Qwen3.8-Flash-Next-UD-IQ4_XS	Distillation model.
DISTILLER_API_KEY	local-sk	Distillation API key.
WORKER_ENDPOINTS	http://localhost:9931/v1, http://localhost:9932/v1	Comma-separated worker base URLs (duplicates removed).
MIN_WORKER_ENDPOINTS	2	Minimum number of distinct worker endpoints; fewer configured or reachable aborts the run.
WORKER_MODEL	Qwen3.8-9B-Q4_K_M.gguf	Worker model name.
WORKER_API_KEY	local-sk	Worker API key.

6.2 Server geometry and context budgets

Variable	Default	Meaning
APEX_SERVER_CTX / APEX_SERVER_NP	65536 / 1	Mirror of apex -c / -np.
WORKER_SERVER_CTX / WORKER_SERVER_NP	196608 / 2	Mirror of worker -c / -np (smallest node if heterogeneous). Per-slot window: 98,304 tokens.
WORKER_PARALLEL_SLOTS	= WORKER_SERVER_NP	Slots used per worker, clamped to WORKER_SERVER_NP.
CHARS_PER_TOKEN	3.5	Character-to-token conversion used for all budgets.
APEX_MAX_OUTPUT_TOKENS	8192	Ceiling for apex outputs.
APEX_GEN_TOKENS	4096	Phase 1 generation length.
APEX_PLAN_TOKENS	4096	Phase 3a partition output (a JSON array).
APEX_DISTILL_TOKENS	8192	Phase 2/6 distillation output.
APEX_POLICY_TOKENS	4096	One policy revision.
APEX_RESERVE_TOKENS	2048	Safety margin in the apex window.
MAX_CONTEXT_CHARS	60000 (clamped)	Apex input ceiling in characters.
MAX_CHUNK_CHARS	40000 (clamped)	Chunk size for Phase 0 batching and splitting.

Variable	Default	Meaning
WORKER_INPUT_CHARS	90000 (clamped)	Total agent prompt size in characters.
MAX_WORKER_TOKENS	8192 (clamped, ≥ 1024)	Agent output ceiling.
WORKER_RESERVE_TOKENS	2048	Safety margin in the worker window.
WORKER_TIMEOUT_SECS	300	Per-request client timeout.
WORKER_MIN_DECODE_TPS	4.0	Used to derive $WORKER_MAX_WALL_SECS = \max(600, MAX_WORKER_TOKENS/tps + 120)$.
MAX_DECOMPOSE_TASKS	20	Upper bound on roster size N .

6.3 Agent prompt sub-budgets

The agent input window `WORKER_INPUT_CHARS` is split into four hard sub-budgets, so that no single section can starve the others:

Variable	Default share	Section
AGENT_CONTEXT_BUDGET	35 %	Broader context (the distilled task list), orientation only
AGENT_ROSTER_BUDGET	15 %	Team roster rendered as explicit exclusions
AGENT_COMMS_BUDGET	35 %	Team communication digest (§10)
AGENT_OBJECTIVE_BUDGET	15 %	The agent’s own objective
AGENT_PARENT_BUDGET	70 % of context	For continuations: inherited deliverables, carved <i>out of</i> the context budget
COMMS_PEER_SUMMARY_CHARS	700	Truncation length per teammate log in the digest

6.4 Recursive self-improvement

Variable	Default	Meaning
RSI_ROUNDS	3	Default for $-n$ (outer iterations t). Round 1 writes; rounds 2 and 3 continue the code.
RSI_MAX_ROUNDS	12	Upper bound for $-n$.
ROUND_BUDGET	15	Agent calls per round (fixed, independent of the roster size).
ROUND_BUDGET_PER_TASK	0 (off)	If > 0 : budget = $\text{round}(N \times \text{this})$ instead of <code>ROUND_BUDGET</code> .
ROUND_BUDGET_MIN / ROUND_BUDGET_MAX	3 / 60	Clamp for the budget. A budget below N is warned about.
CARRY_FORWARD	1	In round 2+, each assignment’s first new branch continues its best earlier attempt (§17.8).
LINEAGE_PATIENCE / LINEAGE_EPS	2 / 0.005	Continuations in a row without a q improvement larger than ϵ before a line of work is DEAD (§17.8).
PI0_BRANCH_FRACTION	0.34	Share of assignments (weakest first attempts) for which π_0 opens a second root.
DREAM_MIN_HEADROOM	0.01	Skip the apex revisions if the replay oracle bound beats the deployed policy by less (§7.9).
INHERIT_POLICY	1	Start round 1 from the newest earlier dream winner in the same category (§17.11).
REBUTTAL_CALLS	4	Budget of the rebuttal round after the final skeptic review; 0 disables it (§17.10).
MAX_PARALLELISM	endpoints W $\times$ slots	W : the maximum batch size of one decision round, online and in replay.
ONLINE_MAX_DECISION_ROUNDS	32	K_1 : decision-round cap of an online rollout.
REPLAY_MAX_DECISION_ROUNDS	$= K_1$	K_2 : decision-round cap of a replay.
DREAM_CANDIDATES	3	$M - 1$: chained revisions per offline phase. $M = \text{this} + 1$ versions are evaluated.
DREAM_BETA1	0.002	β_1 : cost per revealed node in Eq. (1).
DREAM_BETA2	0.004	β_2 : weight of the parallelism bonus $N / \max(1, k^*)$ in Eq. (1).
DREAM_TRACE_CHARS	6000	Size of the per-round replay trajectory digest given to the policy-development agent.

Variable	Default	Meaning
SUPPORT_PROBE_FRAC	0.0	<i>Extension, off by default.</i> Fraction of each round’s budget reserved for off-policy support probes (§7.6).

6.5 Evaluator

Variable	Default	Role (§8.1–8.2)
EVAL_W_STATUS	0.35	Weight for a successful status
EVAL_PARTIAL_STATUS_FRAC	0.5	Fraction of status credit for a salvaged, truncated output
EVAL_W_FILES	0.25	Weight for having at least one deliverable
EVAL_W_LOG	0.15	Weight for a <code><log></code> (full credit at ≥ 200 chars, half credit if shorter)
EVAL_W_NOVELTY	0.25	Weight $\times$ fraction of emitted files whose content hash is new
EVAL_W_VIOLATION	0.20	Penalty per violation (at most 3 counted)
EVAL_W_TRUNCATED	0.15	Penalty for truncation
EVAL_HEURISTIC_MIX	0.6	α : heuristic share of the evaluator score
EVAL_INLINE_TESTS	1	Generate and execute unit tests for every attempt as part of its evaluation
EVAL_MAX_TEST_FILES	6	At most this many testable files per attempt are tested
EVAL_UNTESTED_PRIOR	0.5	Pass-rate term for an attempt with deliverables but no executed test
RECONCILE_MIN_DUP_CHARS	64	Files shorter than this (normalized) are ignored as duplicate candidates
EVAL_INTEGRATION / EVAL_INTEGRATION_MIX	1 / 0.5	Check each attempt inside a project of every sibling’s best deliverable; share of the whole-project score q in s_v (§8.2)
EVAL_CREDIT / EVAL_CREDIT_MIX / EVAL_CREDIT_GAIN	1 / 0.5 / 2.0	Blend q with the attempt’s own contribution (leave-one-out delta, §17.4)
ENFORCE_RUN_IN_INTEGRATION	1	Add the run group (RUN + negative control + executed modules) to every attempt’s integration (§17.4)
INTEGRATION_IMPORT_SECS / INTEGRATION_PYTEST_SECS	30 / 180	Integration time limits
INTEGRATION_CMD / INTEGRATION_CMD_SECS	"" / 300	Optional extra integration command (same as <code>--integration-cmd</code>)
EVAL_DEP_REJECT_SCORE	0.0	Score of an attempt that imports a module the container does not have (§17.3)
EVAL_SHORTCUT_CAP	0.2	Score cap for an attempt that sets metrics from a control flag or to constants (§17.3)
EVAL_MISUSE_CAP	0.3	Score cap for an attempt calling library members that do not exist or with wrong arguments (§17.3)

6.6 Policy sandbox

Variable	Default	Limit
POLICY_MAX_CHARS	20000	Policy source length
POLICY_MAX_RPC	4000	All interface calls, reads included (safety net against polling loops)
POLICY_CPU_SECS	20	RLIMIT_CPU of the policy child process
POLICY_MEM_MB	512	RLIMIT_AS of the child
POLICY_REPLAY_WALL_SECS	60	Wall-clock cap per replay (live rollouts have none, because waiting on agents costs the child no CPU)

Batch size and rollout length are governed by W , K_1 and K_2 (§6.4), not by sandbox limits.

6.7 Phase 5 (unit tests)

The unit tests are part of the evaluator (§7.8). These variables control their generation and execution.

Variable	Default	Meaning
TEST_PIP_INSTALL	1	Install the sanitized requirements*.txt of each evaluated attempt into tests/.venv
TEST_PIP_ALLOWLIST	""	Comma list. If non-empty, only these distributions may be installed
TEST_CPU_SECS / TEST_MEM_MB / TEST_FSIZE_MB	60 / 4096 / 128	ulimit for compilation and test execution
MAX_OUTPUT_TOKENS	4096 (clamped)	Test-generation output ceiling
TEST_MIN_DECODE_TPS	10.0	Derives TEST_TIMEOUT_SECS = max(300, MAX_OUTPUT_TOKENS/tps + 60)
TEST_STALL_SECS	20	Test generation streams; no data for this long is a stall, retried on another worker endpoint (§17.13)

6.8 Phase 0 (Git intake)

Variable	Default	Meaning
GIT_ALLOW_PRIVATE_HOSTS	0	Permit LAN hosts (e.g. self-hosted Gitea). If off, private, loopback, link-local, CGNAT and unresolvable hosts are refused.

6.9 Hard-coded constants (edit the source to change)

Constant	Value	Meaning
GIT_CLONE_DEPTH / GIT_CLONE_TIMEOUT	1 / 600 s	Shallow single-branch clone
REPO_MAX_FILE_BYTES	200,000	Skip larger files
REPO_MAX_TOTAL_CHARS	4,000,000	Global ingestion cap
REPO_MANIFEST_MAX_ENTRIES	400	Manifest listing length
REPO_SUMMARY_REDUCE_DEPTH	3	Maximum recursive reduce passes
MAX_RETRIES / WORKER_RETRIES	3 / 3	Apex and agent retry counts
Agent sampling	$T = 0.4$, freq. 1.1, pres. 0.5	run_agent
Partition / policy-dev sampling	$T = 0.7 / 0.8$	Apex
Test-gen sampling	$T = 0.1$, top-p 0.95, freq. 0.5, pres. 0.2	Phase 5
Integration group weights	coverage 0.20, compile 0.10, import 0.15, pytest 0.35, command 0.20, run 0.40	q is the weighted mean over the groups that apply
run group weights	runs 0.5, control 0.3, executed modules 0.2	§17.4
Packaging tooling excluded from discovery	pip, setuptools, wheel, distribute, pkg-resources	§17.2

Sampling penalties are sent by default and dropped automatically for endpoints that reject them.

6.10 Contract, container and verification

Variable	Default	Meaning
SYNTHESIZE_CONTRACT	1	Synthesize DELIVERABLES and CONSTRAINTS from the prompt text when the brief has no pinned section (§17.1)
SYNTHESIZE_INTERFACES	1	Let the planner write INTERFACES, RUN, PROBE, ABLATIONS and KNOWN ANSWERS when the contract has no INTERFACES (§17.1)
AGENT_BRIEF_BUDGET	min(12000, 12 %)	The original prompt, verbatim, in every agent's input (carved out of the context budget)
ENFORCE_DEPENDENCIES	1	Discover the container's modules and reject attempts that import anything else (§17.2)

Variable	Default	Meaning
ENV_RESCAN_EACH_ROUND / ENV_SECTION_BUDGET	1 / 6000	Rescan before every round; size of the CONTAINER ENVIRONMENT block
API_FACTS_BUDGET / API_FACTS_MAX_MODULE_NAMES	6000 / 150	Size of the LIBRARY API FACTS block; modules with more public names are only checked name by name
QRACK_LIB_PATH	/usr/local/lib/qrack/libqrack_pinvoke.so	Exported for PyQrack probes and required in every file that imports PyQrack
FREEZE_INTERFACES / AGENT_API_BUDGET	1 / 8000	Freeze the integrated project's public API after each round (§17.7)
RUN_COMMAND / RUN_COMMAND_SECS / AGENT_RUN_BUDGET	contract / 300 / 5000	Grounding command (overrides the contract's RUN), its time limit, and the size of the run output shown to agents
PROBE_COMMAND	contract	Negative-control command (overrides the contract's PROBE)
PROBE_IGNORE_KEYS	time elapsed runtime seconds duration timestamp seed pid	Numeric keys ignored when comparing run and control output
FINAL_SKEPTIC_REVIEW / REVIEW_CODE_CHARS	1 / 36000	Apex review of the final run's metrics and control (§17.10)
FINAL_WRITEUP_REFRESH	1	Rewrite the .md deliverables against the final run (§17.10)

6.11 Transport resilience

Variable	Default	Meaning
TRANSIENT_RETRIES	4	Retries of a model call after HTTP 429/5xx, "high demand", "overloaded", dropped connections
TRANSIENT_BACKOFF_SECS	5, 15, 45, 90	Backoff schedule (with jitter); a shutdown request interrupts the wait
TEST_STALL_SECS	20	See §6.7
MIN_WORKER_ENDPOINTS	2	See §6.1

7. Workflow: phase-by-phase

7.1 Phase 0: Git repository intake (-g)

1. **Validation.** The URL is matched against an allowlist of URL shapes. Every resolved address of the host is checked, and resolution failure is treated as blocked (fail-closed).
2. **Clone.** `git clone --depth 1 --single-branch --no-recurse-submodules` runs with `http.followRedirects=false`, `protocol.file.allow=never`, `protocol.ext.allow=never` and `GIT_TERMINAL_PROMPT=0`. The clone goes to a temporary directory that is removed on exit (`atexit`).
3. **Collection.** Files are walked, restricted by `--git-path` if given, and filtered by extension or special filename. Vendored and build directories are excluded (`node_modules`, `vendor`, `build`, `third_party`, ...), as are lockfiles, symlinks, empty files, files over 200 kB, and binaries (a NUL byte in the first 8 KiB). READMEs and manifests are ordered first, then shallow paths. Files longer than `MAX_CHUNK_CHARS` are split into numbered parts.
4. **Embedding or map-reduce.**
 - If header plus sources fit in `MAX_CONTEXT_CHARS`, the sources are embedded verbatim with language-tagged fences.
 - Otherwise the files are packed into batches of at most `MAX_CHUNK_CHARS`. The batches are summarized in parallel on the worker pool (a per-file audit prompt, optionally biased by `--focus`). The summaries are then reduced recursively (at most 3 passes). The reduction stops early if a pass compresses by less than 5 %, and the result is truncated to budget.
5. **Output.** `<timestamp>_git-repo-analysis-<name>_<hex6>.md`, containing the source URL, branch, HEAD commit, ingestion statistics and file manifest.

7.2 Phase 1: generation and Phase 2: distillation (apex)

- **Phase 1** (for -p/-f) streams a comprehensive Markdown document on the prompt topic ($T = 0.7$, `APEX_GEN_TOKENS`) into `<timestamp>_<slug>_<hex6>.md`.
- **Phase 2** reads the Phase 0 or Phase 1 document, truncated to `MAX_CONTEXT_CHARS`, and extracts only actionable TO-DOs and requirements ($T = 0.3$) into `<raw-stem>_distilled.md`.

The distilled document is the *query* that the rest of the pipeline works on. It also serves as every agent’s “broader context”.

7.3 Phase 3a: partitioning and roster (apex)

The partitioner prompt requires between 3 and 20 **mutually exclusive** assignments, each naming the concrete deliverable it owns. Work that must touch the same artefact is merged into one assignment. The output must be a flat JSON array of strings. It is extracted by a bracket-balanced scanner and validated. If there are more than `MAX_DECOMPOSE_TASKS` entries, the list is clamped. If three attempts fail, the whole query becomes a single assignment.

`build_roster` assigns ids `t01...tNN` and directories `tNN_<slug>` (the first four words of the objective). The roster is persisted to `comms/roster.json` and exported to `tasks/tNN.md`. On resume, the roster is **loaded, never re-planned**, so task identities stay stable across rounds, which replay requires.

7.4 Round budget

Let N be the roster size. The per-round budget B (in discovery-agent calls) is

$$B = \text{clamp}(B_0, 3, 60), \quad B_0 = \begin{cases} \text{--budget} & \text{if given} \\ \text{round}(N \cdot \text{ROUND_BUDGET_PER_TASK}) & \text{if that is } > 0 \\ \text{ROUND_BUDGET} = 15 & \text{otherwise.} \end{cases}$$

The fixed default keeps the cost per round predictable across prompts. With carry-forward (§17.8), a nine-assignment roster spends nine calls on continuations of each assignment’s best attempt and six on extra branches and fixes. A budget below N is warned about, because some assignments would receive no attempt.

The policy may spend $B_\pi = B - R$, where R is the reserve of the optional support-probe extension:

$$R = \max(0, \min(\text{round}(\text{SUPPORT_PROBE_FRAC} \cdot B), B - N)).$$

With the paper-aligned default `SUPPORT_PROBE_FRAC=0`, $R = 0$ and $B_\pi = B$. The same B_π caps every replay during dreaming, so the online and offline budgets are aligned. The budget is a resource cap on top of the decision-round caps K_1/K_2 . It mirrors the identical per-round budgets under which [1, §4] compares Dream-RSI with Recursive Fixed Exploration.

Worked examples (defaults):

Roster	Budget B	Policy budget B_π
$N = 7$	15	15
$N = 9$	15	15
$N = 20$	15 (warned: below N)	15
$N = 9$, <code>ROUND_BUDGET_PER_TASK=2</code>	18	18

7.5 Phase 3b: the online rollout (workers)

`run_online_round` first rescans the container (§17.2), then creates a `LiveExplorer` with root node `rRRn0000`, budget B_π , batch limit W and decision-round cap K_1 . It then executes the round’s policy file `policy/pi_rRR.py` in the sandbox (§9.4). If the deployed policy fails (validation error, crash or resource kill), the event is logged and π_0 continues on the **remaining** budget and decision rounds. This is a robustness measure; the stop reason records it.

Decision rounds and legality (after [1, §3]):

- **Legal actions.** At any moment, the legal actions are $A(\mathcal{T}) = \{(\text{root}, t) : t \in \text{roster}\} \cup \{(v, \text{task}(v)) : v \text{ a leaf}\}$. A root action opens a new independent branch for assignment t . A leaf action continues that branch. A node that already has a child is no longer continuable, so every non-root node has at most one child.
- **Decision rounds.** One `ctx.expand_parallel(C)` call is one decision round. C may hold at most W distinct actions that are legal in the tree as it stood before the call. Inadmissible requests (illegal, duplicate or beyond W) return `None` and cost nothing. If no request is admissible, no round is consumed.
- **Termination.** The rollout ends when the policy returns (the empty batch), when K_1 rounds have been completed, or when the budget is exhausted.
- **Concurrency.** The admissible actions of a round run concurrently on the worker pool. The policy observes all their outcomes before its next decision.

Anatomy of one attempt (`LiveExplorer._run_one` $\rightarrow$ `run_agent` $\rightarrow$ `evaluate_node_inline`):

1. **Reserve.** One budget unit is reserved. Each retry (up to `WORKER_RETRIES`) reserves another unit and is refused once the budget is exhausted, so `cost = number of real agent calls`.
2. **Continuation.** For a leaf action, the leaf's deliverables are copied into the new node directory and rendered in the prompt. The agent emits only the files it changes or adds. In round 2 and later, the **first** root action per assignment is also a continuation: of that assignment's best earlier attempt (carry-forward, §17.8), with that attempt's evaluation placed in the prompt. Further root actions are fresh attempts.
3. **Prompt assembly.** The prompt concatenates, each fitted to its budget:
 - the **roster**, rendered as "assignments owned by other agents — do not produce these";
 - the **broader context** and the **team communication digest**;
 - the **original user prompt, verbatim**;
 - the **contract** (pinned or synthesized, §17.1), the **CONTAINER ENVIRONMENT** and **LIBRARY API FACTS** blocks (§17.2), the **CURRENT INTERFACES** frozen after the last round (§17.7) and the **LATEST REAL RUN** of the integrated project (§17.5);
 - the **stage note**, which carries the evaluation of the attempt being continued, the **previous attempts at this assignment, findings routed to this agent** (§17.9) and, for test owners, the **required test cases** from the known answers;
 - the **objective**.
4. **Streaming call.** The worker call streams with a wall-clock guard (`WORKER_MAX_WALL_SECS`). `finish_reason == "length"` or the wall clock marks the attempt as truncated.
5. **Parsing.** A single-pass scanner splits the output into `<file path="...">`, `<log>` and `<note to="tNN">` blocks, skipping tags inside fenced or inline code:
 - **unbalanced tags** $\rightarrow$ `failed_validation`, and no files are written;
 - **truncated output with complete files** $\rightarrow$ `partial`: the complete files are salvaged;
 - **output shorter than 20 characters** $\rightarrow$ `failed_validation`.
6. **Path resolution.** Declared paths are rebuilt *inside* the node directory, so escape is impossible by construction:
 - absolute paths or `..` are recorded as violations;
 - a path naming a teammate's directory is recorded as a violation and written under `claimed/`;
 - paths are truncated to their last three components.
7. **Novelty.** Each emitted file's normalized SHA-256 is compared against the parent's hashes (continuation) or this assignment's hashes from earlier rounds (fresh attempt).
8. **Evaluation.** For a successful or partial attempt, the fixed evaluator runs on the same worker slot, in this order:
 1. the **dependency gate** (§17.3): an import the container cannot satisfy rejects the attempt (score 0, nothing tested);
 2. the heuristic of §8.1 plus the inline unit tests of §7.8;
 3. **integration** inside a project of the siblings' best deliverables, including the per-attempt **run check** and **credit** (§17.4);
 4. the **lineage verdict** (§17.8);
 5. the **shortcut** and **library-misuse** scans, which cap the score (§17.3);
 6. the whole evaluation is appended to the attempt's log (§17.8).
 The resulting score s_v is final.
9. **Recording.** The node is appended to `trees/roundRR.jsonl`, logged to `comms/events.jsonl`,

and its log is written to `comms/roundRR/<node>_<task>.md`. Notes from accepted outputs are delivered to the recipients' inboxes. For continuations, $\text{gain} = s_v - s_{\text{parent}}$.

At the end of the rollout, the per-round decision trace (batch composition, rejected requests, revealed scores, running best, spend) is written to `comms/roundRR/online_trace.json`.

7.6 Off-policy support probes

Extension; not part of [1]; disabled by default (`SUPPORT_PROBE_FRAC=0`).

When enabled, a reserve R is withheld from the policy. After the policy returns, the explorer's budget is raised by exactly R . Calls the policy chose not to spend are not handed to the probes. The probes use the same legality and W -batch rules as the policy, in two interleaved lists:

- **deep:** continue each assignment's best leaf that has deliverables;
- **wide:** open a further root branch for the assignments with the fewest branches.

Motivation. Replay can only serve continuations that some logging policy actually took. Probes widen the support of the recorded pool beyond the incumbent's own choices. If you enable them, they are applied in both arms (dream and `--no-dream`), so budgets stay equal. Probe nodes carry `"support": true`, and their decision rounds are marked in the trace.

7.7 Mechanical reconciliation

`reconcile_round` walks the best node per task and hashes every file (ignoring trivial boilerplate such as `__init__.py` and licence files, and files under `RECONCILE_MIN_DUP_CHARS`). It writes `comms/roundNN/RECONCILE.md` containing:

- coverage (tasks reached versus never expanded), failed best attempts, truncations;
- a best-attempt-per-agent table;
- **duplicate work:** identical content owned by more than one agent. The first-listed agent retains ownership and the others are instructed to reference its path;
- **filename collisions** across agent directories;
- **scope violations**.

This report is injected into the next agents' communication digests and pulled explicitly into Phase 6.

7.8 Phase 5: unit tests inside the fixed evaluator

In [1, §3], a fixed evaluator scores each candidate as part of its generation-evaluation request and returns diagnostic feedback. The script realizes the test component of that evaluator inline (`evaluate_node_inline`, enabled by `EVAL_INLINE_TESTS=1`).

1. **Selection.** The attempt's testable files (`.py`, `.c`, `.h`, `.cpp`, `.hpp`, `.cc`, `.cxx`, `.sh`, `.bash`) are selected, including inherited ones, up to `EVAL_MAX_TEST_FILES`.
2. **Generation.** A test is generated on the worker slot the attempt already holds, using a terse test-writer prompt that also receives the contract, the container environment and the library API facts. The call streams: if no data arrives for `TEST_STALL_SECS` (20 s), it is abandoned and retried on another worker endpoint (§17.13). It is cached per round by (*content hash, filename, language*), so an unchanged inherited file reuses its test but is re-executed against the new sibling files. For C/C++, the test must `#include` the artefact and supply its own `main()`. Any artefact `main()` is renamed to `autoresearch_artifact_main()`, and header tests link a sibling implementation.
3. **Environment.** Tests run in a run-scoped venv `tests/.venv`. The attempt's own `requirements*.txt` are installed after sanitization: plain name[extras] <specifiers> lines only, binary wheels only, optional allowlist. Installations are serialized, and already-installed lines are skipped. The environment is minimal, with no inherited API keys or endpoints.
4. **Execution** under `ulimit` CPU/memory/file-size limits in a separate process group:
 - `pytest`: 45 s;
 - `bash`: 30 s;
 - C/C++: compile at 60 s, run at 30 s.

Outcomes are `PASSED`, `FAILED`, `COMPILE_ERROR`, `TIMEOUT`, `SKIPPED` or `ERROR`. The pass rate is taken per test file.

5. **Scoring and diagnostics.** The node’s final score is $s_v = \alpha h_v + (1 - \alpha)\rho_v$ (§8.2). The diagnostics `test_pass_rate`, `test_count`, `tests_passed` and up to five `test_failures` are stored on the node and shown to the policy.
6. **Reports.** At the end of the round, all executions are written to `reports/execution_report_roundRR.json`. The cumulative `reports/execution_report.json` and `.csv`, which Phase 6 reads, are updated.

Test generation is an evaluator call, not a discovery-agent call, and is not charged to the budget. It does occupy the worker slot, so wall-clock time per attempt grows with the number of testable files. Setting `EVAL_INLINE_TESTS=0` reduces the evaluator to the heuristic and the untested prior.

A `.round_complete` marker is written once the rollout, reconciliation and reports are complete. A round without it is treated as interrupted on resume (§12.3).

7.9 Dreaming: offline policy improvement (apex)

`dream_policy_improvement` implements the offline phase of [1, §3]. It runs after **every** round, including the last, so that π_{t+1} always exists and a later `-r -n t+1` continues the lineage.

1. **Fixed history.** $\mathcal{H}_t$ is every `trees/roundRR.jsonl`. It does not change during the phase.
2. **Version 0.** The deployed policy $\pi_t^0 = \pi_t$ is replayed from the root of every recorded tree (§8.4) and scored by Eq. (1) (§8.3).
3. **Headroom gate.** The replay oracle bound V^* is the best value any policy could reach with hindsight on the recorded pool: an exact multiple-choice knapsack over assignments, respecting replay’s reveal order. If $V^* - V^0 < \text{DREAM_MIN_HEADROOM}$ (0.01), no revision can win meaningfully and the $M - 1$ apex calls are skipped (`--force-dream` overrides). Note that a policy which spends its full budget on its own recorded tree always reaches the oracle’s *quality*; headroom then comes only from pruning or from trees with real branching.
4. **Chained revisions.** For $m = 0, \dots, M - 2$, the policy-development agent (apex) receives:
 - the interface and objective specification;
 - the **source of version** m ;
 - its replay scores, per tree and averaged;
 - its **per-decision-round replay trajectories**: batch composition (new branches versus refinements), rejected requests, revealed scores, running best, spend and stop reason;
 - the scores of all earlier versions;
 - a compact summary of the recorded history.

It revises the **best valid version so far** (not necessarily the last one) into version $m + 1$, which is saved to `dream/roundRR/candidate_MM.py` and replayed on the same history. An invalid version (static rejection, sandbox kill, crash) is listed with its error, and the next revision is asked to repair it. A version that leaves assignments without an attempt that the recorded pool could have reached is **invalid** (“covers only 44% of the assignments”), however cheap it looks in V .

5. **Fresh π_0 as a candidate.** Whenever the deployed policy differs from the current default π_0 (because it was dreamed or inherited), π_0 is replayed as an extra candidate at zero agent cost, so a bad lineage is corrected.
6. **Selection.** $\pi_{t+1} = \pi_t^{m^*}$ with $m^* \in \arg \max_m V^m$. Ties go to the earliest version, so the deployed policy is retained on a tie.
7. **Persistence.** The winner is written to `policy/pi_r(t+1).py` with a provenance header that records its replay score against the deployed version’s (only such dream *winners* are ever inherited by later runs, §17.11). All scores go to `dream/roundRR/scores.json`, all replay trajectories to `dream/roundRR/policy_execution_traces.jsonl`, and a dream event is logged.

Replay costs **zero agent calls**. The wall-clock cost of a dream is dominated by $M - 1$ serial apex generations plus $M \times |\mathcal{H}_t|$ sandboxed replays.

Coverage guard. After the dream, if the policy deployed in the round just finished left any assignment without an attempt, the next round’s policy file is overwritten with π_0 and `[RSI] ... pi_0` restored is printed. This is a live safety net independent of replay.

7.10 Final stages, run manifest and Phase 6

After the last round, three stages run before the manifest (§17.10):

1. a **skeptic review** of the final run’s metrics and negative control (apex);
2. a **rebuttal round**: the review’s findings, together with the round’s routed findings, go to the owning assignments, which get up to REBUTTAL_CALLS agent calls as continuations of their best attempts; the project is then re-integrated, re-run and re-reviewed;
3. a **write-up refresh**: the owner of each .md deliverable rewrites it against the final run output and the review, and every decimal in it is checked against the output.

RUN_MANIFEST.md is a mechanical index containing: the query, the roster, per-round statistics (policy file, agent calls, decision rounds, nodes, mean best score, duplicates, violations), the dreaming table (versions, selected version, its replay score, the deployed version’s score, whether the policy changed), the full discovery-tree table, a deliverable index, and token and endpoint statistics.

Phase 6 then asks the distiller to turn the raw intake, the reconciliation reports and the test telemetry into DISTILLED_TASKS.md. Failed tests and ownership conflicts become high-priority TODOs, with embedded code or traceback excerpts. Agent deliverables (work/), trees, policies and dream logs are deliberately excluded from Phase 6 input. With --iterate, an existing DISTILLED_TASKS.md is refined against the new telemetry instead of being regenerated.

Finally the token ledger is summarized (§17.12): a console table, TOKENS.md, tokens_summary.json, and the same tables appended to RUN_MANIFEST.md.

8. Evaluator and replay objective

8.1 Heuristic component

For a node v with status σ_v , deliverable set F_v , violation list Λ_v , truncation flag τ_v , log length ℓ_v and novel-file fraction $\nu_v \in [0, 1]$, the heuristic is

$$h_v = \text{clip}_{[0,1]} \left(w_s \mathbb{1}_s(\sigma_v) + w_f \mathbb{1}[F_v \neq \emptyset] - w_x \min(|\Lambda_v|, 3) - w_t \tau_v + w_\ell \lambda(\ell_v) + w_n \nu_v \right),$$

where the status and log indicators are

$$\mathbb{1}_s(\sigma) = \begin{cases} 1 & \sigma = \text{success} \\ \text{EVAL_PARTIAL_STATUS_FRAC} & \sigma = \text{partial} \\ 0 & \text{otherwise} \end{cases} \quad \lambda(\ell) = \begin{cases} 1 & \ell \geq 200 \\ 0.5 & 0 < \ell < 200 \\ 0 & \ell = 0. \end{cases}$$

With the default weights $(w_s, w_f, w_\ell, w_n) = (0.35, 0.25, 0.15, 0.25)$, a clean, successful, fully novel attempt with a substantive log attains $h_v = 1$.

8.2 Fixed evaluator score

The evaluator is applied once, at creation, and its score is final:

$$s_v = \alpha h_v + (1 - \alpha) \rho_v, \quad \rho_v = \begin{cases} \text{pass rate of the attempt's executed tests} & \text{if at least one test ran} \\ \text{EVAL_UNTESTED_PRIOR} & \text{if } F_v \neq \emptyset \text{ but no test ran} \\ 0 & \text{if } F_v = \emptyset, \end{cases}$$

with $\alpha = \text{EVAL_HEURISTIC_MIX}$. This is the *file-level* score f_v . With integration enabled (the default), the final score also reflects the whole project the attempt belongs to (§17.4):

$$s_v = (1 - \lambda_I) f_v + \lambda_I ((1 - \lambda_C) q_v + \lambda_C c_v), \quad c_v = \text{clip}_{[0,1]}(0.5 + g(q_v^{\text{full}} - q_{\text{base}}^{\text{full}})),$$

where q_v is the weighted mean of the integration groups (coverage, compile, import, pytest, optional command, and the run group), c_v the credit for this attempt’s own contribution against its assignment’s previous accepted best, $\lambda_I = \text{EVAL_INTEGRATION_MIX}$, $\lambda_C = \text{EVAL_CREDIT_MIX}$ and $g = \text{EVAL_CREDIT_GAIN}$. A first attempt and a test-only attempt get the neutral $c_v = 0.5$. Finally the gates of §17.3 apply: a rejected attempt scores $\text{EVAL_DEP_REJECT_SCORE}$ (0), a shortcut caps s_v at EVAL_SHORTCUT_CAP (0.2), and library misuse at EVAL_MISUSE_CAP (0.3).

Because s_v is never rewritten, the score the online policy observed is identical to the score replay reveals. This is exactly the setting of [1, §3], where “scores follow a fixed task-scoring protocol”. Changing the evaluator between runs changes the scale of s_v , so trees recorded before and after such a change should not share a pool (start a fresh run rather than resuming).

8.3 Replay objective (Eq. 1 of [1])

For policy version π^m replayed on recorded world $\mathcal{T}_i$, let $\hat{\mathcal{T}}_i^m$ be the revealed subtree, N_i^m the number of revealed non-root nodes, and $k_i^{m,*}$ the number of completed decision rounds. The script computes

$$V_i^m = \underbrace{\frac{1}{N_{\text{roster}}} \sum_{t=1}^{N_{\text{roster}}} \max \left(s_r, \max_{v \in \hat{\mathcal{T}}_i^m, \text{task}(v)=t} s_v \right)}_{\text{discovery quality}} - \underbrace{\beta_1 N_i^m}_{\text{execution cost}} + \underbrace{\beta_2 \frac{N_i^m}{\max\{1, k_i^{m,*}\}}}_{\text{parallelism bonus}}, \quad s_r = 0,$$

and scores the version by its mean over the fixed history, $V^m = \frac{1}{t} \sum_{i=1}^t V_i^m$.

The cost and parallelism terms are those of [1, Eq. 1], with $\beta_1 = \text{DREAM_BETA1}$ and $\beta_2 = \text{DREAM_BETA2}$. The quality term is [1]’s maximum over revealed nodes, applied per assignment and averaged over the roster; the root (empty workspace) contributes $s_r = 0$. For a single-assignment roster it reduces exactly to [1]’s term.

scores.json reports, per version and per tree: V , quality, N , k , N/k , coverage, agent-call cost and stop reason.

8.4 Replay semantics

ReplayExplorer presents the same interface as the live explorer. It differs only in how a continuation produces its observation:

- **Deterministic transition.** Continuing v with assignment t reveals $\text{Child}(v; \mathcal{T}_i)$, the first not-yet-revealed recorded child of v for t , in recording order. Its stored score and diagnostics become observable, and its recorded cost is charged against B_π .
- **Legality.** An action whose child was never recorded is not legal in replay. `ctx.legal_actions()` lists exactly the recorded continuations, as in the reference policy API of [1, App. B.2]. A request for an unrecorded continuation returns `None` and costs nothing.
- **Termination.** A replay ends when the policy returns (the empty batch), when no admissible request remains, when K_2 rounds have been completed, or when the budget is exhausted.
- **Independence.** Every policy-tree pair is replayed from the root. No revealed state carries over between evaluations.

8.5 Monotonicity

Because version 0 is the deployed policy and ties go to the earliest version, the selected policy satisfies $V^{m*} \geq V^0$ on the fixed history $\mathcal{H}_t$. As in [1, §3], this is a guarantee about average replay score on recorded history, not about future online performance.

9. Writing exploration policies

9.1 Contract

A policy is a Python module that defines exactly one entry point, `explore(ctx)`. Helper functions and module-level constants are permitted. The same source is executed online and in replay.

9.2 The `ctx` interface

Call	Returns	Notes
<code>ctx.tasks()</code>	<code>list[str]</code>	Assignment ids, e.g. ['t01', 't02'].
<code>ctx.root()</code>	<code>str</code>	Id of the shared tree root.
<code>ctx.max_parallelism()</code>	<code>int</code>	W : the largest batch one decision round may hold.
<code>ctx.rounds_left()</code>	<code>int</code>	Decision rounds remaining (K_1 online, K_2 in replay).
<code>ctx.legal_actions()</code>	<code>list[[parent_id, task]]</code>	Actions legal right now. In replay: only recorded continuations.
<code>ctx.expand_parallel([(parent_id, task), ...])</code>	<code>list</code> , aligned 1:1	One decision round. Inadmissible requests return <code>None</code> at no cost.
<code>ctx.expand(parent_id, task)</code>	<code>node</code> or <code>None</code>	A decision round with a batch of one.
<code>ctx.frontier()</code>	<code>list[node]</code>	The current leaves.
<code>ctx.nodes()</code>	<code>list[node]</code>	Every node revealed so far in this rollout.
<code>ctx.best()</code>	<code>node</code> or <code>None</code>	Highest score so far.
<code>ctx.best_per_task()</code>	<code>dict[task, node]</code>	Best node per assignment.
<code>ctx.budget_left()</code> , <code>ctx.spent()</code>	<code>int</code>	Agent-call units.
<code>ctx.note(msg)</code>	<code>None</code>	Diagnostic string (≤ 200 chars, ≤ 200 notes). The first 20 notes appear in replay traces.

A **node** as seen by a policy has the fields `id`, `parent`, `task`, `depth`, `score` (the final evaluator score, in $[0,1]$), `gain`, `status`, `cost` and `files`. It also carries `diagnostics = {violations, truncated, emitted, inherited, tests_passed, tests_total, test_failures, integration_delta, integration_delta_known, lineage_stalls, lineage_dead, lineage_verdict}`. All numeric fields are always numbers (never `None`): `integration_delta` is `0.0` with `integration_delta_known = False` for a first or test-only attempt. The lineage fields are mirrored at the top level of the node, so `n["lineage_verdict"]` and `n["diagnostics"]["lineage_verdict"]` both work. `lineage_verdict` is `GOOD` (the project runs and its control responds), `ENGINEER` (still improving), `DEAD` (`LINEAGE_PATIENCE` continuations without improvement — open a fresh branch instead) or `BAD` (rejected by a gate).

Semantics worth internalizing:

- **Legality is evaluated before the call.** `(root, t)` opens a new branch for assignment `t`. `(leaf_id, task_of_leaf)` continues a branch. A node that has a child is not continuable. At most one new branch per assignment can be opened per decision round, because a batch is a set of distinct actions.
- **Stopping is explicit.** Returning from `explore` is the empty batch, which ends the rollout.
- **Retries are charged.** `cost` may exceed 1.
- **Diagnostics distinguish failure types.** A failed test or a truncated output is often repairable by continuing the leaf; a scope violation usually is not.

9.3 Static rules (enforced before execution)

`validate_policy_source` parses the source with `ast` and rejects it if any of the following holds:

- the source exceeds `POLICY_MAX_CHARS`, or there is no top-level `explore`;
- a top-level statement is anything other than a `def`, an assignment, or a constant expression (docstring);
- it contains any `import`, `async` construct, `global` or `nonlocal`;
- it accesses any attribute beginning with `_`, or interpreter-internal attributes (`gi_frame`, `f_globals`, `tb_frame`, `co_code`, `mro`, ...);
- it uses the names `open`, `exec`, `eval`, `compile`, `__import__`, `getattr`, `setattr`, `type`, `object`, `super`, `print`, `vars`, `dir`, ..., or any name beginning with `_`;

- it contains a bare except: (it would swallow the step-cap abort);
- it contains a string constant containing `__`.

The only builtins available are: `abs all any bool dict divmod enumerate filter float int len list map max min pow range reversed round set sorted str sum tuple zip isinstance frozenset next iter slice hash ord chr repr format callable`, the constants `True/False/None`, and a handful of exception types (including `AttributeError, RuntimeError, LookupError, NotImplementedError`).

9.4 Runtime isolation

The policy runs in a separate interpreter (`python -I -S`) with a clean environment (`PATH, LANG` only) in a temporary working directory and its own session. Before it reads any policy source, the child applies these limits:

- `RLIMIT_CPU: POLICY_CPU_SECS;`
- `RLIMIT_AS: POLICY_MEM_MB;`
- `RLIMIT_FSIZE = 0, RLIMIT_CORE = 0, RLIMIT_NPROC = 0;`
- `RLIMIT_NOFILE = 3` (no new files, sockets or pipes).

The child communicates with the parent **only** through line-delimited JSON RPC restricted to the method names of §9.2. Every return value is a plain JSON projection, so no object graph can be traversed.

Exceeding `POLICY_MAX_RPC` raises an abort that the child re-raises as a `BaseException` subclass, which cannot be caught by `except Exception`. A policy that keeps calling after the abort is killed after 50 further calls. A watchdog enforces `POLICY_REPLAY_WALL_SECS` during replay and honours shutdown requests.

An aborted policy still counts as a *valid* run: its detail string reads `aborted: ...`, and whatever it expanded before the abort is scored.

9.5 The baseline policy π_0

The hand-written baseline is the *parallel refine* strategy used as the common starting point in [1, §4]:

1. It opens one branch per assignment from the root, in batches of at most W . In round 2 and later these first branches are carry-forward continuations of each assignment's best earlier attempt (§17.8).
2. It opens a **second root** for the weakest `PI0_BRANCH_FRACTION` (0.34) of assignments, by first-attempt score, so that recorded trees contain real sibling-versus-continuation choices for replay to learn from.
3. Every following pass continues the current leaf of every open branch, **lowest score first**, in batches of at most W . A leaf whose line of work is **DEAD** is not continued: its assignment gets a fresh branch instead.
4. It stops when the budget or the decision rounds run out, or when no branch can be continued (in replay: nothing further was recorded).

Each branch thus maintains its own local trajectory. Both the dreaming arm and the `--no-dream` control start from π_0 , so round 1 is identical by construction and later divergence is attributable to dreaming.

9.6 Example: a gain-aware adaptive policy

The following policy passes `validate_policy_source` and replays correctly in the sandbox. Each decision round, it builds one batch of at most W legal actions: new branches for assignments not yet reached, then continuations of leaves whose last step gained more than a threshold, best first. It stops once no such action remains. Because it draws only from `ctx.legal_actions()`, it never submits an inadmissible request, online or in replay.

```
# Example: gain-aware adaptive policy (illustrative).
PATIENCE_GAIN = 0.01
MAX_DEPTH = 4
```

```

def worth_refining(n):
    if n["depth"] >= MAX_DEPTH or not n["files"]:
        return False
    g = n["gain"]
    return g is None or g > PATIENCE_GAIN

def explore(ctx):
    w = ctx.max_parallelism()
    root = ctx.root()
    while ctx.budget_left() > 0 and ctx.rounds_left() > 0:
        legal = ctx.legal_actions()
        reached = set(ctx.best_per_task().keys())
        leaves = {}
        for n in ctx.frontier():
            leaves[n["id"]] = n
        opens = [a for a in legal if a[0] == root and a[1] not in reached]
        refines = [a for a in legal if a[0] in leaves and worth_refining(leaves[a[0]])]
        refines.sort(key=lambda a: -leaves[a[0]]["score"])
        batch = (opens + refines)[:min(w, ctx.budget_left())]
        if not batch:
            break
        ctx.expand_parallel(batch)

```

To deploy a hand-written policy for round k , place it at `policy/pi_rKK.py` in the run directory before resuming (§12.6). It then becomes version 0 of the next offline phase.

9.7 Design guidance

The policy-development prompt encodes these heuristics. They are equally useful when writing policies by hand:

1. Allocate depth where continuation has been paying (positive gain) and breadth where it has not.
2. Cover the whole roster. An unreached assignment contributes only the root's score (0) to the quality term.
3. Batch independent continuations up to W per round. Serial expansion lowers the parallelism bonus N/k and leaves worker slots idle.
4. Stop lines that plateau. Every revealed node costs β_1 whether or not it improves anything.
5. Be adaptive: conserve calls while progress is good, and spend them when it plateaus. [1, §5.2] reports this conserve-then-re-expand pattern emerging in learned policies. Under Eq. (1), each revealed node costs β_1 , while full batches earn up to $\beta_2 W$ per tree.
6. Draw batches from `ctx.legal_actions()`. It is the only way to know, in replay, which continuations exist. Rejected requests in a trace (`rejected illegal=...`) indicate a policy that is guessing.
7. Use the diagnostics. Continuing a leaf whose failure is repairable (failed tests, truncation) is often worth more than abandoning the branch.

10. Inter-agent communication

Agents never communicate synchronously; no agent can address another during a call. Communication is **asynchronous, file-mediated, and delivered at prompt-construction time** by `build_comms_digest`. Within `AGENT_COMMS_BUDGET`, the digest contains the following sections in priority order:

Priority	Section	Source	Content
1	Messages addressed to you	comms/roundRR/notes/tNN.md, rounds r...1	Directed notes (<note to="tNN">) from teammates. Delivered only from accepted outputs; an unknown recipient is a violation.
2	The attempt you are continuing	Up to 3 ancestor logs ($\leq 1,400$ chars each)	Lineage of a continuation, with the instruction to improve rather than restart.
3	Reconciliation report	Latest RECONCILE.md	Duplicates, collisions, violations, coverage.
4	What teammates reported this round	Best node per other task	First COMMS_PEER_SUMMARY_CHARS characters of each <log>.

Operational consequences:

- **Siblings in one batch cannot see each other.** The digest is built from a snapshot of the live node list taken immediately before the call. Agents dispatched in the same `expand_parallel` batch observe each other's outputs only from the next expansion onwards.
- **Delivery depends on the policy.** A note is read only if the policy expands the recipient again. A note to a task the policy never revisits is never read, and there is no reply or acknowledgement channel.
- **Inboxes accumulate.** Notes are appended per round and never cleared. Because the inbox has the highest priority, stale notes can crowd out lineage, reconciliation and peer sections in long runs.
- **Only the head of a log propagates.** Peer summaries keep only the start of each <log>, so agents should state dependencies and open questions first.
- **Structured context by design.** The digest deliberately contains raw, structured context rather than abstracted directional advice. `--semantic-guidance` prepends such advice only to reproduce the negative result of [1, §5.1].

11. Run directory layout

```

<dir>/<category>/run_<YYYYmmdd_HHMMSS>_<hex6>/
|-- <ts>_<slug>_<hex6>.md           Phase 0/1 raw intake
|-- <ts>_<slug>_<hex6>_distilled.md  Phase 2 query
|-- CONTRACT.md                     Pinned or synthesized contract (§17.1)
|-- seeds.jsonl                     Round-0 seed attempts (--seed-from only)
|-- tasks/tNN.md                     One file per assignment
|-- work/tNN_<slug>/<node_id>/...    Agent deliverables, one dir per node
|   |-- claimed/...                  Quarantined cross-agent paths
|-- comms/
|   |-- roster.json                  Frozen roster (ids, dirs, objectives)
|   |-- brief_meta.json              Contract, deliverables, verbatim brief, run/probe
↪ commands, ablations
|   |-- events.jsonl                 Append-only structured event log
|   |-- partition_failures/attempt_N.txt Raw planner reply of a failed partition attempt
|   |-- roundRR/
|       |-- <node_id>_<task>.md       Per-attempt log (header + <log> body)
|       |-- notes/tNN.md             Inbox for agent tNN
|       |-- RECONCILE.md              Mechanical reconciliation
|       |-- online_trace.json         Decision rounds of the online rollout
|       |-- .round_complete           Completion marker (after rollout + reports)
|-- trees/roundRR.jsonl              Discovery tree (the replay world)
|-- policy/pi_rRR.py                  Policy deployed in round RR
|-- dream/roundRR/candidate_MM.py    Policy version MM (revision of version MM-1)
|-- dream/roundRR/scores.json         Replay results for all versions
|-- dream/roundRR/policy_execution_traces.jsonl Replay trajectories (version x world)
|-- env/roundRR.json                  Container scan: distributions, modules, devices (§17.2)

```

```

|-- env/roundRR_api_facts.md          Library API facts shown to agents that round
|-- integration/roundRR/              Integrated project of the round's best attempts
|-- integration/roundRR_api.{json,md} Frozen public API (§17.7)
|-- integration/roundRR_run.{json,md} Grounding run, control, ablations, origins (§17.5)
|-- integration/roundRR_routed.json   Findings routed to owners (§17.9)
|-- integration/latest/              Final deliverables (incl. the refreshed write-up)
|-- integration/final_review.md       Skeptic review (§17.10)
|-- integration/final_writeup_refresh.json Refresh result and unmatched numbers
|-- tests/.venv/                     Run-scoped test environment
|-- tests/roundRR/<task>/<node>/test_* Evaluator tests of each attempt
|-- tests/roundRR/grounding*/        Throwaway copies used by grounding, control and
↳ ablation runs
|-- reports/execution_report*.{json,csv}
|-- aborted/roundRR_<ts>/...         Artefacts of an interrupted round
|-- tokens.jsonl                     One record per model call (§17.12)
|-- tokens_summary.json, TOKENS.md   Token, runtime and throughput summary
|-- RUN_MANIFEST.md
`-- DISTILLED_TASKS.md

```

Node ids have the form `rRRnSSSS` (round, sequence), and `rRRn0000` is the round's root. Trees are append-only JSONL and a node's score is never rewritten. The root record also stores the W in force when the tree was recorded.

12. Standard operating procedures

12.1 Single exploratory run

```

export OPENAI_API_BASE=http://apex:9933/v1
export WORKER_ENDPOINTS=http://w1:9931/v1,http://w2:9932/v1
python3 autoresearch-rsi-onefile.py -p "Design and implement a Vulkan-backed statevector kernel
↳ library" -n 1

```

With `-n 1`, one online round is executed and tested. A dream still follows, producing `pi_r02.py`, so the run can later be extended without loss.

12.2 Multi-round recursive run

```
python3 autoresearch-rsi-onefile.py -f spec.md -n 5 --budget 40
```

Each round deploys the policy selected by the previous dream. Monitor the [DREAM] lines: for each version they report V , quality, N , k , N/k and coverage, and finally which version was selected.

12.3 Resuming after interruption

```

python3 autoresearch-rsi-onefile.py -r          # same category, newest run
python3 autoresearch-rsi-onefile.py -r -c mycat # a specific category

```

Resume skips every phase whose artefact exists. A round lacking `.round_complete` is **archived** to `aborted/` before being re-run from a clean slate, because node ids restart at `rRRn0000` and must not collide. If the policy for the next round is missing, it is dreamed from the recorded pool before the round starts.

The first SIGINT/SIGTERM requests a graceful stop between work items. A second forces an exit.

12.4 Extending a finished run

```
python3 autoresearch-rsi-onefile.py -r -n 6 # after 5 completed rounds, run round 6
```

12.5 Dreaming without agents

```
DREAM_CANDIDATES=8 python3 autoresearch-rsi-onefile.py -r --dream-only
```

This replays the deployed policy and a chain of 8 revisions over the full pool, and overwrites the next round's policy file with the argmax. Only the apex needs to be online. It is useful for spending idle apex time on policy search while the worker pool is busy or down.

12.6 Injecting a hand-written policy

1. Validate the policy offline:

```
import importlib.util
spec = importlib.util.spec_from_file_location("ar", "autoresearch-rsi-onefile.py")
ar = importlib.util.module_from_spec(spec); spec.loader.exec_module(ar)
print(ar.validate_policy_source(open("my_policy.py").read())) # -> (True, 'ok')
```

2. Replay it against the recorded pool, to check that it is legal and to read its trajectories:

```
from pathlib import Path
run_dir = Path("run_data/projects/run_<id>"); src = open("my_policy.py").read()
pool = ar.load_pool(run_dir); tasks = [r["id"] for r in ar.load_roster(run_dir)]
res = ar.replay_score(src, pool, tasks, budget=40)
print(res["score"]); print(ar._trace_digest(res, 4000))
```

3. Copy it to policy/pi_rKK.py, where KK is the next round to run.
4. Resume with -r -n KK.

12.7 Controlled comparison against fixed exploration

Replicate the paper's primary control [1, §4] at equal budget:

```
python3 autoresearch-rsi-onefile.py -f spec.md -n 5 --budget 40 -c arm_dream
python3 autoresearch-rsi-onefile.py -f spec.md -n 5 --budget 40 -c arm_fixed --no-dream
```

Compare RUN_MANIFEST.md per-round *mean best score* against cumulative agent calls, which mirrors the quality-compute trajectories of [1, Fig. 3b]. Two confounds must be controlled:

- **Planning and generation.** Phases 1–3a are re-run in each arm, so the two arms may receive different rosters. For a strict comparison, share one roster:
 1. Run the dreaming arm.
 2. Copy its run directory into the second category.
 3. In the copy, keep only the raw and distilled .md files, tasks/ and comms/roster.json.
 4. Resume the copy with -r -c arm_fixed -n 5 --budget 40 --no-dream.
- **Sampling.** Agent outputs are stochastic ($T = 0.4$). Use several seeds or runs per arm.

12.8 Semantic-guidance ablation

```
python3 autoresearch-rsi-onefile.py -f spec.md -n 5 --budget 40 -c arm_guided
↪ --semantic-guidance
```

This tests the §5.1 claim of [1] in this setting.

12.9 Repository-driven work

```
python3 autoresearch-rsi-onefile.py -g https://github.com/twobombs/thereminq-examples \
  --git-path qft --focus "semiclassical Shor scaling and OpenCL portability" -n 3
```

For a LAN-hosted forge, additionally set `GIT_ALLOW_PRIVATE_HOSTS=1`.

12.10 Tuning checklist

Symptom	Adjustment
Many partial / truncated nodes	Raise worker <code>-c</code> and <code>WORKER_SERVER_CTX</code> , or lower <code>WORKER_INPUT_CHARS</code> , so <code>MAX_WORKER_TOKENS</code> is not clamped.
Rounds are slow	Inline tests occupy worker slots. Lower <code>EVAL_MAX_TEST_FILES</code> , or set <code>EVAL_INLINE_TESTS=0</code> for exploratory runs (keep it fixed within an experiment).
Dreams never change the policy	Increase <code>DREAM_CANDIDATES</code> (longer revision chains). Grow the pool with more rounds. Optionally enable <code>SUPPORT_PROBE_FRAC</code> to widen replay support.
Policies spend the full budget without gain	Increase <code>DREAM_BETA1</code> .
Policies expand serially	Increase <code>DREAM_BETA2</code> ; check W in the [BUDGET] block.
Rollouts end on the round cap	Raise <code>ONLINE_MAX_DECISION_ROUNDS</code> (and <code>REPLAY_MAX_DECISION_ROUNDS</code>).
Low coverage	Raise <code>ROUND_BUDGET</code> , or set <code>ROUND_BUDGET_PER_TASK</code> .
Stalls dominate unit-test time	Lower <code>TEST_STALL_SECS</code> ; check the gateway (both ports usually reach the same upstream).
Dreams always skip (“headroom below epsilon”)	Expected with small, self-recorded pools. Raise <code>PI0_BRANCH_FRACTION</code> for more branching, run more rounds, or use <code>--force-dream</code> .
The same bug survives rounds	Read <code>integration/roundRR_routed.json</code> and the attempt logs’ evaluation sections; lower <code>LINEAGE_PATIENCE</code> to abandon dead lines sooner.
<code>q</code> is high but the review calls metrics invalid	Inspect <code>integration/roundRR_run.md</code> (control verdict, ablations, NOT EXECUTED) and the shortcut events; see §17.6.
Replay rejects everything	Inspect <code>dream/roundRR/scores.json</code> for the static-validation reason (§15).

Keep β_1 , β_2 , W , K_1/K_2 and the evaluator settings fixed across all rounds and arms of an experiment. They define the objective and the evaluator, which [1] holds fixed.

12.11 Short prose prompts

A prompt without INTERFACES / CONSTRAINTS / ACCEPTANCE CRITERIA works: the contract forge (§17.1) synthesizes deliverables and constraints from the prompt text only, and the planner adds a shared API, a run command, a negative control, ablations and known answers. Check the start-up lines:

```
[CONTRACT] Synthesized DELIVERABLES, CONSTRAINTS, DEPENDENCIES, INTERFACES, RUN, PROBE,
  ↪ ABLATIONS, KNOWN ANSWERS, RUN RULES, TEST RULES
[DELIVERABLES] 7 required: ...
[RUN] grounding command after each round: python3 runner.py ...
[PROBE] negative control after each successful run: python3 runner.py --corrupt ...
  [!] Dropped known answer (keyed to an experimental condition - that is the outcome being
  ↪ measured): ...
```

A few pinned lines in the prompt (a CONSTRAINTS block, a DELIVERABLES list) override the synthesis for those sections.

12.12 Compounding across runs

- **Policy inheritance** is automatic (`INHERIT_POLICY=1`): a new run in the same category starts round 1 with the newest earlier dream *winner*, printed as `[RSI] round 01 policy inherited from ...`. Set `INHERIT_POLICY=0` for a clean baseline.

- **Project seeding** is explicit: `--seed-from latest` (or a run directory name) continues the previous run's final project instead of rewriting it. Use it for iterative frontier expansion on the same prompt, in the spirit of [15, §4.3].

```
python3 autoresearch-rsi-onefile.py -f spec.md --seed-from latest
```

12.13 Hardening runs with error-prone models

Point all three ports at a gateway for a fast, cheap model; its failures are the test data. Useful habits:

- run the same prompt repeatedly and upload the run directory after each run; the console log alone is not enough to find root causes;
- read `integration/roundRR_run.md` first (does it run, does the control respond, what never executed), then `integration/roundRR_routed.json`, then `integration/final_review.md`;
- when switching models, expect different sampling-penalty and context quirks; the transport layer handles the common ones (§17.13).

12.14 Reading the token summary

`TOKENS.md` separates *average generation rate* (token-weighted), *median per call, throughput while calls are in flight* (parallel slots counted) and *throughput over the whole runtime*. A large gap between average and median points at stalls; the unit-test generation (stalled) row shows how much time they cost. Agent attempts typically read 7–10 prompt tokens per generated token, so prompt context is the lever for cost.

13. Correspondence with Dream-RSI

13.1 Mapping

Dream-RSI [1, §3]	This script
Discovery agent (fixed)	Worker-tier agent with the <code>_PROMPT_PHASE3_AGENT</code> contract
Fixed evaluator with diagnostics, score s_v fixed at creation	Heuristic + inline unit tests, applied once per attempt (§8.1–8.2); diagnostics stored on the node and observable by the policy
Root r , <code>Continue(v)</code>	Shared round root <code>rRRn0000</code> ; (<code>root</code> , <code>t</code>) opens a branch for assignment t , (<code>leaf</code> , <code>task</code>) continues it; a continuation resumes the parent's workspace (files)
$A(\mathcal{T}) = \{r\} \cup \text{leaves}$	<code>legal_actions()</code> ; legality enforced before each call; non-leaves are not continuable
Batch C , $ C \leq W$	<code>expand_parallel</code> = one decision round; at most <code>MAX_PARALLELISM</code> distinct legal actions
$C = \emptyset$ terminates; K_1 online, K_2 replay	Returning from explore; <code>ONLINE_MAX_DECISION_ROUNDS</code> , <code>REPLAY_MAX_DECISION_ROUNDS</code>
Online rollout, stochastic transition	<code>LiveExplorer</code> ; each admissible action runs one agent attempt and the evaluator
History $\mathcal{H}_t$	<code>trees/round*.jsonl</code>
Replay: deterministic reveal of <code>Child(v; $\mathcal{T}_i$)</code> , prefix-only observation	<code>ReplayExplorer</code> ; only recorded continuations are legal; the policy sees only revealed nodes
Every policy-tree pair replayed from the root	<code>replay_tree</code> constructs a fresh explorer per pair
Eq. (1): $\text{quality} - \beta_1 N + \beta_2 N / \max(1, k^*)$;	<code>replay_value</code> , <code>replay_score</code> (§8.3)
$V^m = \frac{1}{t} \sum_i V_i^m$	
Versions $\pi_t^0 = \pi_t, \pi_t^1, \dots, \pi_t^{M-1}; \pi^{m+1}$ revised from π^m using its trajectories, scores and earlier feedback	<code>dream_policy_improvement</code> with <code>DREAM_CANDIDATES = M - 1</code> (§7.9)
$\pi_{t+1} = \arg \max_m V^m$, hence $V^{m^*} \geq V^0$	<code>_select_winner</code> (ties $\rightarrow$ earliest version)
Only the exploration-policy code is updated	Models, prompts, evaluator and interfaces are fixed; policies are Python in a sandbox

Dream-RSI [1, §3]	This script
Recursive Fixed Exploration baseline, same budget and initialization	--no-dream; both arms start from the parallel-refine π_0
Prompt-level semantic guidance ablation (§5.1)	--semantic-guidance
Reference API (App. B.2): legal_actions, max_parallelism, probe_batch, execution traces between rounds	ctx.legal_actions, ctx.max_parallelism, ctx.expand_parallel, policy_execution_traces.jsonl

13.2 Remaining adaptations

The following differences remain. They stem from the multi-assignment setting, from local operation, or from details that [1] leaves to the reference implementation.

1. **Many assignments per tree.** [1] formulates one discovery problem per tree. Here all N assignments share one root per round. Root actions therefore name their assignment, and discovery quality is averaged over the roster (§8.3). With $N = 1$, this reduces exactly to [1].
2. **Resource budget in agent calls.** In addition to K_1/K_2 , each rollout is capped at B_π discovery-agent calls, with retries charged. [1, §4] runs both arms under identical per-round budgets; the cap enforces that here.
3. **Evaluator content.** [1] uses task-specific scoring protocols (benchmark runtimes, objective values, correctness checks). Here the fixed evaluator is a generic contract/novelty heuristic combined with auto-generated unit tests (§8), because assignments are open-ended build tasks. The policy sees file paths and diagnostics, but not deliverable bodies.
4. **Inadmissible requests.** [1] requires a legal batch. The script accepts a mixed batch, executes its admissible part, and returns None for the rest at no cost. A fully inadmissible batch consumes no round.
5. **Reference-prompt features not implemented.** The reference prompt of [1, App. B.2] also prescribes a swept beta exploration knob with a Pareto-AUC evaluation, and a plan_grid method that sets the width and depth of the next live grid. These belong to the authors' reference controller rather than to the formalism of §3, and are not implemented.
6. **Robustness fallback.** If the deployed policy crashes online, π_0 completes the rollout on the remaining budget and rounds. The stop reason records the fallback.
7. **Support probes** (§7.6) are an optional extension, off by default.
8. **Legacy trees.** Trees recorded by earlier revisions of the script are normalized on load: their creation-time score becomes the node score. Their non-leaf re-expansions simply become unreachable in replay.
9. **Carry-forward roots.** In round 2 and later, each assignment's first root action continues its best earlier attempt (§17.8). The tree structure is unchanged (the node is still a child of the round root), so replay semantics are unaffected; only the online transition differs from [1]'s fresh root.
10. **Richer evaluator.** The fixed evaluator now includes integration, a run check with a negative control and executed-module coverage, credit for the attempt's own contribution, and score caps for shortcuts and library misuse (§8.2, §17.3-17.4). It is still fixed at creation, so replay stays exact.
11. **Dream safeguards.** The headroom gate, the coverage validity check, revision from the best valid version, the fresh- π_0 candidate and the online coverage guard (§7.9) are additions to [1]. They never change which of two valid versions wins on V .
12. **Cross-run inheritance.** [1] improves a policy within one discovery run. Inheriting the newest dream winner into a new run (§17.11) extends the lineage across runs.
13. **Elements adopted from ScientistTwo [15].** Per-line-of-work verdicts with pruning, strict-improvement updates of the integrated version, ablations as component checks, a review-driven rebuttal round, and iterative expansion from a previous result (--seed-from) follow the corresponding stages of [15, §3.2-3.5, §4.3], reduced to the scale of this pipeline.

14. Security model

The script executes model-written code in two places and treats both as untrusted.

- **Exploration policies:** AST validation, a separate interpreter with resource limits, no file descriptors, and an RPC-only surface (§9.3–9.4).
- **Agent deliverables under test:**
 - a run-scoped venv, sanitized requirements, and binary wheels only (`--only-binary=all:`, so no `sdist` build scripts run);
 - an optional allowlist;
 - a minimal environment without inherited keys or endpoints;
 - `ulimit` CPU, memory and file-size limits;
 - process-group kills on timeout.

Agent file writes are confined to the node directory by path reconstruction. Phase 0 enforces fail-closed SSRF checks, disables redirects and the `file/ext` transports, and skips submodules.

These controls are defence in depth, **not** a container boundary. They do not isolate the network for Phase 5 test execution, and `RLIMIT_NPROC=0` may behave differently for privileged users. Consistent with local-only agent operation, run the pipeline inside a resource- and egress-metered container.

15. Diagnostics and troubleshooting

Message / symptom	Cause	Remedy
Agent tier failed smoke tests. Aborting.	At least one worker endpoint failed <code>ping_tier</code> .	Fix or remove the endpoint from <code>WORKER_ENDPOINTS</code> ; every listed worker must answer.
UNDER-PROVISIONED: node window ... < budgeted per-slot ...	A server's <code>n_ctx</code> is smaller than the script assumes.	Align <code>*_SERVER_CTX/*_SERVER_NP</code> with the real <code>-c/-np</code> .
Agent output budget collapsed ... 1024 floor	The input window leaves no room for output.	Lower <code>WORKER_INPUT_CHARS</code> or raise the server context.
Deployed policy failed (...). Falling back to <code>pi_0</code>	The policy was rejected statically or killed at runtime.	Read the detail. Common causes: imports are not permitted, blocked name <code>'print'</code> , bare <code>'except:'</code> , CPU limit.
<code>pi_M</code> : invalid in replay (...) in a dream WARNING: <code>pi_rKK.py</code> not found; deploying <code>pi_0</code> rejected <code>illegal=...</code> in traces	The same checks, applied to a revision. The dream lineage is broken (e.g. a deleted file). The policy requests continuations that are not legal (non-leaves, cross-assignment, or unrecorded in replay).	Normal. Invalid versions cannot win; the next revision is asked to repair it. Run <code>-r --dream-only</code> before resuming. Draw batches from <code>ctx.legal_actions()</code> .
stop: decision-round cap reached Rounds much slower than before unbalanced output tags ... deliverables not written	The rollout hit K_1 (or K_2). Inline evaluator tests occupy worker slots. The worker model violated the tag contract.	Raise the cap or batch more per round. Lower <code>EVAL_MAX_TEST_FILES</code> or disable inline tests for exploratory runs. Check the model and chat template. Reduce prompt pressure (smaller sub-budgets).
note addressed to unknown agent <code>'tXX'</code> Round RR was interrupted; moved ... to aborted/... Resume failed: No valid run directories Phase 5 <code>COMPILE_ERROR</code> everywhere	An agent addressed a non-existent teammate. The resume found an incomplete round. The category contains no <code>run_*</code> directory with <code>.md</code> artefacts. Missing compiler, or the artefact is not self-contained.	Informational. Counted as a violation. Expected. The round is re-run cleanly. Check <code>-d/-c</code> . Install <code>gcc/g++</code> . Inspect <code>tests/roundRR/</code> .

Message / symptom	Cause	Remedy
Agent pool has 1 endpoint(s) ... at least 2 are required smoke test failed: ... models/<name> is not found HTTP 400 ... Penalty is not enabled for this model transient error (503: ... high demand); retry k/4 in Ns	Fewer than MIN_WORKER_ENDPOINTS worker endpoints configured. The gateway maps to an upstream model that does not exist. The endpoint rejects sampling penalties. Upstream overload.	Add a second worker port or lower MIN_WORKER_ENDPOINTS. Fix the gateway's model name; the run stops before any work. Informational: penalties are dropped and remembered for that endpoint. Informational: retried with backoff (§6.11).
unit-test generation on ...: no data for 20s; retry ... on ...	A streamed test-generation call stalled.	Informational: retried on another endpoint.
rNNnXXXX (tYY) rejected: imports qiskit_aer	An import the container cannot satisfy (also full dotted paths).	Expected gate behaviour; install the package if it should be available (the gate rescans before rejecting).
calls missing library members: ... -> score capped at 0.3	A member or argument count that the installed library does not have.	Expected; the attempt's log names the nearest real members.
shortcut: experiment.py:NN ... -> score capped at 0.2	A metric set from a control/ablation flag or to a constant.	Expected; see §17.6.
[PROBE] ... HARDCODED / SCORE-ONLY / INSENSITIVE	The negative control does not move the measured quantities by measurement.	Read the routed findings (§17.9).
[ROUTE] tNN: ...	A project-level finding sent to the owner who must act.	Informational.
Interfaces need revision (attempt 1/2): ...	The planner's INTERFACES do not thread the PROBE flag or do not pin key formats.	Informational: one retry with the problems listed.
Dropped known answer (...)	A known answer prescribed the score, a flag, a run mode or an experimental condition.	Informational.
[RSI] round NN policy covered only k/N assignments; pi_0 restored N number(s) in report.md match nothing in the run output	The deployed policy skipped assignments.	Informational: coverage guard.
No devices found. Check OpenCL installation! in run output	The write-up quotes numbers the run did not print. PyQrack found no OpenCL device (it falls back to CPU).	Inspect integration/final_writeup_refresh.json. See the COMPUTE DEVICES block of env/roundRR.json; enable an OpenCL ICD (e.g. RUSTICL_ENABLE=<driver>, a vendor ICD, or POCL).

Primary data sources for post-hoc analysis:

- comms/events.jsonl (Appendix B);
- trees/round*.jsonl (Appendix A);
- dream/round*/scores.json;
- integration/round*_run.md, integration/round*_routed.json, integration/final_review.md;
- tokens.jsonl, TOKENS.md;
- RUN_MANIFEST.md.

16. Known limitations

1. **Replay only covers where history went.** Replay is exact on recorded branches, and only recorded continuations are legal in it. Policy improvement is therefore bounded by the support of the recorded pool, which is why the procedure must be a loop.
2. **The evaluator is still a proxy.** The heuristic rewards contract compliance, novelty and the presence of deliverables; the tests are auto-generated. The verification layer (§17) makes cheap shortcuts expensive: constant metrics, controls that do not move, invented APIs,

hollow known-answer tests. It cannot prove that a measurement is *physically* right. The strongest signal remains a known-answer test from first principles, and the planner can still get those wrong.

3. **Stochastic online transitions.** Online outcomes are stochastic, as in [1]. Replay-score gains need not translate into online gains in a single round.
4. **Small pools, little headroom.** With three rounds and DREAM_CANDIDATES=3, dreams operate on one to three trees. A policy that spends its whole budget on its own recorded tree already reaches the oracle's quality, so headroom comes only from pruning and branching; expect frequent "headroom below epsilon" skips. Policy inheritance (§17.11) is what lets improvements accumulate across runs.
5. **Evaluator cost.** Inline tests add wall-clock time per attempt and occupy worker slots, although they are not charged to the agent-call budget.
6. **Communication-channel constraints** (§10). Batch siblings are blind to each other, in-boxes are never pruned, and peer summaries are truncated.
7. **Resume heuristic.** `-r` binds to the most recently modified run in the category. Keep concurrent experiments in separate categories.
8. **Single-apex bottleneck.** Planning, dreaming and distillation are serialized on one apex slot.
9. **ASCII normalization.** Non-ASCII characters are stripped from all model outputs and ingested sources.
10. **Static analysis is heuristic.** The flag trace, consumer chain, instance-method tracking and shortcut scans follow names within and across local modules. Values passed through containers, configuration objects or dynamic dispatch can escape them; in doubt they stay silent rather than guess.
11. **Per-attempt run checks cost time.** Every attempt's integration runs the project twice (run and control) plus a repeat for the noise floor. Cheap for prototypes; set ENFORCE_RUN_IN_INTEGRATION=0 for projects whose run takes minutes.
12. **Unit-test stalls.** Streaming with failover limits the damage, but when the upstream behind a gateway stalls, all ports stall together.

17. Verification and hardening layer

The Dream-RSI loop improves *how* the budget is spent. It takes the evaluator as given, and a weak evaluator is exactly what error-prone models exploit: constant metrics, controls that set their own result, simulators that ignore their input, tests that cannot fail. This section describes the layer that closes those gaps. Every mechanism here is **mechanical** (static analysis, subprocess runs, file checks) unless marked as a model judgment, and every one was introduced after a run in which its absence produced a plausible-looking but empty result (§18).

17.1 Contract forge

The contract is the binding text every agent and every test generator receives. It is assembled once, saved as `CONTRACT.md` and stored in `comms/brief_meta.json` for resume.

1. **Pinned sections first.** If the brief contains INTERFACES, CONSTRAINTS or ACCEPTANCE CRITERIA sections, they are used verbatim, and a numbered DELIVERABLES list is extracted.
2. **Synthesis from the prompt only.** Otherwise the apex writes DELIVERABLES and CONSTRAINTS from the *user's text alone*, never from the Phase-1 draft, which had previously become the de-facto spec (it once chose Cirq, which the container did not have). The result is marked `# SYNTHESIZED CONTRACT`; where it and the prompt disagree, the prompt wins.
3. **DEPENDENCIES** is written from the container scan (§17.2), not from a list.
4. **Planner INTERFACES.** If there is no INTERFACES section, the planner writes one shared API: every public name and complete signature per module, in dependency order. It must pin *value vocabularies* (`Literal[...]` for option strings, exact dict keys, gate-tuple layouts and the complete set of op names), pin *measurement key formats* (bitstring length and qubit order, or declared marginals), and *thread the PROBE flag* as an optional parameter through both the entry function and the experiment function. The output is validated; on problems the planner gets one retry with the problems listed.

5. **RUN, PROBE, ABLATIONS.** RUN is the entry command (it must print a JSON "score" line). PROBE is a negative control: one bare, hyphenated flag (`--corrupt`) that breaks the *process* while the metric's reference stays the original intended state. ABLATIONS are two or three flags that each switch off one component. `--flag True` is normalized to `--flag`, underscores to hyphens.
6. **KNOWN ANSWERS**, validated: a line is dropped if it prescribes the summary score, refers to the control flag, names a run mode (default, corrupted, control, ...), is not of the form `metric: input -> expected`, or is **keyed to an experimental condition** (a value of the experiment function's `Literal[...] parameters`, e.g. forward/reverse). The last rule exists because a planner once stated the outcome under study as a "known answer", and agents hard-coded it.
7. **RUN RULES and TEST RULES** are appended: a run succeeds only with exit 0, a score line and no reported error; failures print a traceback; no catch-all except, no silent fallback; printed prose is not a result; boolean switches use `action='store_true'`. Tests check known answers from first principles and never assert the outcome being measured.
8. **The original prompt, verbatim**, is added to every agent's input (§7.5).

17.2 Container discovery

The container defines the abilities of the workload. Nothing is configured; everything is scanned in the evaluation interpreter (the run-scoped venv with system site-packages), at start-up and again before every round:

- **Distributions and modules.** `importlib.metadata` plus a module-path scan, without importing anything. Packaging tooling is excluded. The result is the CONTAINER ENVIRONMENT block, with new, removed and upgraded packages reported per round.
- **Dependency gate.** An attempt that imports a third-party module outside that set is rejected before any test (score `EVAL_DEP_REJECT_SCORE`, 0.0), including optional imports inside `try`. Full dotted paths are verified too (qiskit installed does not make `qiskit.providers.aer` exist). Before rejecting, the container is rescanned once, so a package the owner installs mid-round counts at once. Agents never install anything; pip installs in the test venv are restricted to discovered packages.
- **Library API facts.** For libraries the prompt names and every library name the integrated project uses, the real API is inspected: classes with complete member lists, signatures, enum values, and the first docstring lines and Args sections of members the project actually calls. Uses that do not exist (`QrackSimulator.rz`, `Statevector.partial_trace`) are listed first as DOES NOT EXIST, including methods called on library *instances* (tracked through `x = Class(...)`, `self.x = module.Class(...)`, with `...` as `x` and class-method constructors such as `Class.from_instruction(...)`). Calls with the wrong number of arguments or unknown keywords are listed as WRONG ARGUMENTS.
- **Compute devices.** OpenCL platforms and devices (`pyopencl` or `clinfo`), Vulkan devices (`vulkaninfo`), `/dev/dri`, and whether PyQrack actually finds an OpenCL device or falls back to CPU. Shown to agents as COMPUTE DEVICES, with the guidance to prefer OpenCL/Vulkan and not to target CUDA.

17.3 Attempt-level gates and caps

All gates are part of the fixed evaluator and run once, at creation. Their findings are violations, appear in the attempt's log, and cap the score:

Gate	Detects	Effect
Dependency	Imports the container cannot satisfy (module or dotted path)	Score 0.0; not tested
Shortcut	A metric (matched by name or name part, e.g. <code>target_fidelity ~ fidelity</code>) assigned inside a branch on the PROBE or any ABLATION flag (constant, cap such as <code>min(x, 0.25)</code> , scale such as <code>x * 0.5</code>); a conditional expression on the flag yielding a constant; a metric set to a non-trivial constant inside <i>any</i> <code>if</code> branch (e.g. per experimental condition). Guards that use 0 or 1 are not flagged	Score capped at 0.2
Library misuse	Members that do not exist on library instances; wrong argument counts or unknown keywords against the real signature	Score capped at 0.3

Gate	Detects	Effect
Frozen interface	Removing a name siblings use, dropping a parameter, adding a required one (§17.7)	Violation

17.4 Integration, run check and credit

Each attempt is placed into a project assembled from every sibling's best accepted deliverable, and the project is checked as a whole. The groups and their weights:

Group	Weight	Measures
coverage	0.20	Required deliverables present
compile	0.10	Python files compile
import	0.15	Modules import
pytest	0.35	Pass rate of the project's tests
command	0.20	Optional --integration-cmd score
run	0.40	The RUN command in a throwaway copy: runs successfully (0.5); the negative control responds by measurement and no metric is set to a constant (0.3: RESPONSIVE 1, PROBE FAILED 0.5, INSENSITIVE, SCORE-ONLY or HARDCODED 0); required modules executed (0.2)

Credit. The whole-project q is computed with the attempt and with its assignment's previous accepted best in its place; the difference, times EVAL_CREDIT_GAIN, around 0.5, is the attempt's own contribution (§8.2). One shared bug no longer gives every attempt the same score, and a fix or a regression stands out. First and test-only attempts get the neutral 0.5 (a correct test that exposes a sibling's bug must not count against its author). Gate-rejected attempts are never a baseline, never integrated and never carried forward.

Strict improvement. The version of an assignment that goes into the integrated project changes only if whole-project q improves (then score); on a tie the incumbent stays, so the project cannot slide backwards.

17.5 Grounding run

After each round, the RUN command executes in a copy of the integrated project:

- **Success** requires exit 0 *and* a "score" line *and* no reported error ("status": "error", an error field, a traceback, a non-numeric score). A runner that catches everything and exits 0 fails.
- **stdout and stderr are separated**, and stdout is split into JSON result blocks and other lines, because libraries print to both (PyQrack's OpenCL notice goes to stdout). Long string fields are flagged as **UNTRUSTED FREE TEXT**: prose the code printed, not a result.
- **Exception origins** are recorded by a sitecustomize hook using sys.monitoring (Python 3.12+): where each exception was raised and which project frames it passed through, **including exceptions the code caught** (this exposed silent fallbacks to a reference simulator).
- **Executed files and functions** are recorded by the same hook. A required module that never ran is reported as NOT EXECUTED.
- The output, control verdict, ablation table and these findings are saved to integration/roundRR_run.md and shown to every agent in the next round.

17.6 Negative control and ablations

After a successful run, the PROBE command and every ABLATION command run in their own copies and are compared with the main run field by field:

- **Noise floor.** The unmodified RUN command is run once more. A value that moves between two identical runs counts as responding only if the control moves it by more than three times that noise, and never below a relative 1e-6 (float jitter once produced a false RESPONSIVE).

- **Verdicts.** RESPONSIVE (a measured quantity moved), INSENSITIVE (nothing moved), SCORE-ONLY (only the summary score moved while the measured quantities did not), HARDCODED (the static scan of §17.3 found metrics set from the flag or to constants; takes priority over the numbers), PROBE FAILED or UNCOMPARABLE.
- **Process, not reference.** The control must break the process (skip the correction gates, break the entangled pair, flip the input *without telling the metric*). A control that also changes the metric's reference cancels itself; the flag trace flags any assignment of the flag into a reference-named variable (expected..., target, ref_, ideal, truth).
- **Ablations** use the same machinery; their table (MOVES THE METRICS / NO EFFECT / ONLY THE SCORE MOVES / HARDCODED / FAILED TO RUN) is the project's ablation study and must be reported by the write-up.

17.7 API freeze

After each round, the integrated project's public API is extracted with ast: functions, classes with constructors and public methods, dataclass fields and constants, each with the files that use it. Imports of names a module does not define and calls with keywords a callee does not accept are listed as UNRESOLVED. The next round sees this as CURRENT INTERFACES, with the rule to keep used names compatible (adding optional parameters is allowed) and to announce changes with a <note>. Breaking a used name is a violation (§17.3).

17.8 Iterating code across rounds

- **Carry-forward.** In round 2 and later, each assignment's first new branch continues the attempt with the highest whole-project q , preferring the most recent on a tie (it has moved past the older error), skipping rejected, shortcut and DEAD attempts; a --seed-from seed serves when there is none. Its files are already in place and its evaluation is placed in the prompt.
- **Evaluation in the log.** After evaluation, each attempt's log receives an *Evaluation* section: score breakdown, change to the project's q , unit-test pass count and failures, whole-project findings per group (including the run check), line-of-work verdict and every violation. Continuations read it through the lineage.
- **Previous attempts.** Every attempt sees the last six attempts of its assignment with where each started and the first failure it hit, headed "do not repeat a change that already failed" (a round once repeated a failed fix verbatim because the attempt that tried it had been forgotten).
- **Line-of-work verdicts.** GOOD, ENGINEER, DEAD or BAD per attempt (§9.2). A line is DEAD after LINEAGE_PATIENCE continuations without a q improvement above LINEAGE_EPS; it is no longer continued or carried forward, and π_0 opens a fresh branch for that assignment instead.

17.9 Routing findings to owners

A project-level failure concerns everyone equally in the scores, so on its own it moves no one. After each grounding run, findings are sent to the specific assignments that must change a file, shown in their next prompts as FINDINGS ROUTED TO YOU, printed as [ROUTE] lines and saved to integration/roundRR_routed.json:

- **Crash origin** → the owner of the file where the exception started. A ModuleNotFoundError for a *project* module goes to the owner of the missing module (with its recent failures), not to the importer.
- **Control-flag trace** (when the control is not RESPONSIVE): from the entry point through local calls, reporting where the flag stops — parsed but never passed on, passed to a function without that parameter, received but never used, used only inside the entry module, or assigned into a reference (self-cancelling). Owners of experiment functions without a flag parameter are told to add an optional one.
- **Insensitive despite wiring:** the values the flag changes (e.g. circuit under if corrupt:) are followed through local calls and methods of project classes to the deepest consumer. When several backends are assigned to the same name, the one that *actually executed* (from the recorded functions) is followed. Both suspects are told: the consumer ("ignores or silently drops parts of its input: fixed result, unknown ops skipped, coarse thresholding")

and the **producer** of the changed value (“check that what you build implements the protocol end to end, so that removing a required step must change the measured result”).

- **Never-executed required modules** (after a successful run only) → at most two run-path modules that should call them (preferring those whose INTERFACES block names the module) and the module’s owner.
- **Hollow known-answer tests** → the test owners, when no assertion both mentions a known-answer metric and compares it with a value near a known answer.

17.10 Final stages: review, rebuttal, write-up

- **Skeptic review** (model judgment, not proof). The apex reads the entry file first, then the rest of the code, the final run output and the control output, traces every printed key to the line that computes it, and returns per metric valid | suspect | invalid with the deciding line, plus CONTROL: genuine | hardcoded | not wired | self-cancelling. Saved to integration/final_review.md.
- **Rebuttal round**. Suspect and invalid metrics are routed to the owner of the evidence file; a CONTROL verdict without evidence goes to the entry module’s owner and the owners of the modules it imports; all of the round’s routed findings are merged in. The named assignments get up to REBUTTAL_CALLS calls through a dedicated rebuttal policy (not dreamed, never inherited), with the normal evaluation, integration and grounding; the review is then repeated.
- **Write-up refresh**. Each .md deliverable’s owner rewrites it against the final run, quoting numbers as printed, reporting the control and ablation verdicts, and turning every suspect or invalid review finding into an explicit caveat. If the run failed, the write-up must say so and state no results. Every decimal is then checked against the run output and the files it wrote (allowing for rounding); unmatched numbers are listed.

17.11 Compounding RSI across runs

- **Policy inheritance**. A new run starts round 1 with the newest earlier dream *winner* in the same projects directory: a policy file whose header shows it beat the version it replaced. Rebuttal policies and unchanged carry-overs are never inherited (an early version inherited a rebuttal policy that targeted a single assignment, and the run stalled; the coverage guard and the fresh- π_0 candidate now catch any such case after one round).
- **Project seeding**. --seed-from latest|RUN_DIR copies an earlier run’s final files as round-0 seed attempts for every assignment whose deliverables all exist there; round 1 continues them.

17.12 Token and throughput accounting

Every model call is recorded in tokens.jsonl: category (named by the calling stage), tier, round, prompt and completion tokens (server-reported, or estimated from characters and marked), wall time, time to first token, generation rate, and whether the output was cut off. The run prints a projection before round 1 and after each round (using measured means and rates), a live tok ... in / ... out, agent ... tok/s progress line, a per-round summary including aggregate throughput, and at the end a table per category with runtime, time with calls in flight, token-weighted average rate, median per call and throughput. TOKENS.md, tokens_summary.json and the manifest carry the same data; a resumed run’s summary covers the whole run.

17.13 Transport resilience

- **Sampling penalties** rejected by an endpoint are dropped and remembered for that endpoint.
- **Transient upstream errors** (429, 5xx, “high demand”, “overloaded”, dropped connections) are retried with backoff 5, 15, 45, 90 s; long-generation timeouts are not retried.
- **Unit-test generation streams**; a stall of TEST_STALL_SECS is abandoned and retried on the worker endpoint with the fewest recent stalls, never the one that just stalled.
- **At least two worker endpoints** are required, and all must pass the smoke test.

18. Change log since the paper-aligned release

Each change is listed with the failure that motivated it. Most were found in hardening runs with fast, error-prone models on one short test prompt (a small teleportation prototype with a reference and a PyQrack simulator).

Change	Motivating failure	§
Dream headroom gate (replay oracle bound), π_0 second roots, default of several rounds	Dreams produced four identical versions on pure-chain trees; apex calls wasted	7.9, 9.5
Contract synthesis from the prompt only; DEPENDENCIES from the container; verbatim prompt to every agent	A short prompt let the Phase-1 draft choose Cirq (not installed) and collapse eight deliverables into two	17.1, 17.2
Planner INTERFACES, API freeze, grounding run, write-up refresh	Modules guessed each other's APIs; write-ups reported numbers no run had produced	17.1, 17.5, 17.7, 17.10
Library API facts, value vocabularies, leave-one-out credit	Invented rz/cnot/get_amp; "apply_h" vs "h"; one shared bug flattened every score	17.2, 17.4
Strict run success (score line, no error), RUN RULES, final refresh, partition-failure log	A runner caught all exceptions and exited 0	17.5
Negative control and skeptic review; known-answer rules	A fidelity that was 1.0 by construction passed every check	17.6, 17.10
SCORE-ONLY verdict, static hard-code scan, untrusted free text, review traces the flag	Controls that set score = 0.5 under the flag; a canned "conclusion" string	17.6
Flag-alias matching; KNOWN ANSWERS validation; device discovery; stdout/stderr split; shortcut penalty	The flag changed names on the way down; the planner's own known answer prescribed the score; OpenCL silently absent	17.1–17.3, 17.5
Container discovery instead of static lists	Static candidate lists do not match what the container's owner installs	17.2
Token ledger, runtime and tok/s totals	No view of where tokens and time go; unit-test stalls invisible	17.12
Numeric node fields, next/iter in the policy sandbox, coverage check for revisions	Revisions crashed on None > 0 and on next; revisions skipped half the assignments	7.9, 9.2, 9.3
Instance-method checks, traceback rule, exception-origin recorder	sim.rz(...) on instances went unchecked; {"error": ...} hid the failing line	17.2, 17.5
Run check in every attempt's integration, misuse cap, dotted-import check, docstrings in API facts, streaming unit-test generation, executed-module recording	q = 0.945 with a placeholder metric, a disconnected control and a dead PyQrack module	17.2, 17.4, 17.5
Ports 9933 (apex) / 9931, 9932 (agents), minimum two agent endpoints; noise floor for the control; stall failover; transient retry	Float jitter read as a response; three stalls on one port; a 503 killed Phase 6	3.1, 17.6, 17.13
Carry-forward, evaluation in logs, 15 calls per round, 3 rounds	Every round rewrote every file from scratch; refines did not know why their parent failed	7.4, 17.8
Previous-attempts history, carry-forward by q then recency, rejected attempts excluded, class-method constructors, routing to owners	A failed fix was repeated; credit was measured against a rejected attempt; project failures moved no one	17.4, 17.8, 17.9
Flag threaded through INTERFACES; pinned key formats; required test cases; missing-module routing; dream coverage check	The contract left no path for the control flag; counts.get("0") vs "000"	17.1, 17.9
Policy inheritance, project seeding, lineage verdicts, strict improvement, ablations, rebuttal round (after ScientistTwo [15])	Improvements did not compound across runs; the review's findings changed nothing	17.8, 17.10, 17.11
Process-not-reference rule, self-cancelling verdict, argument-count checks, bare hyphenated flags, 20 s stall timeout	A control changed the input and the expected state together; r(params[0], q) missing an argument	17.1, 17.2, 17.6
Inheritance only of dream winners; online coverage guard; coverage check relative to the reachable	A rebuttal policy was inherited and starved eight of nine assignments	7.9, 17.11
Consumer-chain and producer routing, executed-function recording	A simulator ignored its input; a circuit was not teleportation at all	17.5, 17.9

Change	Motivating failure	§
Hollow known-answer test detection; broader shortcut scan (caps, name parts, constants per condition); known answers keyed to experimental conditions dropped; ablation flags in the shortcut gate	$\min(\text{fidelity}, 0.25)$ under the flag; asymmetry = 0.8 per direction copied from a “known answer”; tests that only checked keys	17.1, 17.3, 17.9

References

- [1] T. Zheng, X. Wu, Z. Zhang, Z. He, C. Zhang, B. Coleman, R. Wei, D. Bai, H. Liu, R. Liu, X. Wang, Y. Zhuan, W.-C. Kang, R. Xiang, H. Huang, X. Cheng, Y. Guo. *Dream-RSI: Recursive Self-Improvement through Evolving Worlds*. arXiv:2609.14858, 2026. <https://arxiv.org/abs/2609.14858>. Project page: <https://dream-rsi.com> (PDF: <https://dream-rsi.com/assets/dream-rsi.pdf>).
- [2] D. Ha, J. Schmidhuber. *World Models*. arXiv:1803.10122, 2018.
- [3] D. Hafner, T. Lillicrap, J. Ba, M. Norouzi. *Dream to Control: Learning Behaviors by Latent Imagination*. arXiv:1912.01603, 2019.
- [4] D. Hafner, J. Pasukonis, J. Ba, T. Lillicrap. *Mastering Diverse Domains through World Models*. arXiv:2301.04104, 2023.
- [5] R. S. Sutton. *Integrated Architectures for Learning, Planning, and Reacting Based on Approximating Dynamic Programming*. In *Machine Learning Proceedings 1990*, pp. 216–224. Morgan Kaufmann. doi:10.1016/B978-1-55860-141-3.50030-4.
- [6] T. M. Moerland, J. Broekens, A. Plaat, C. M. Jonker. *Model-based Reinforcement Learning: A Survey*. *Foundations and Trends in Machine Learning* 16(1):1–118, 2023.
- [7] A. Novikov et al. *AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery*. arXiv:2506.13131, 2025.
- [8] S. Liu et al. *EvoX: Meta-Evolution for Automated Discovery*. arXiv:2602.23413, 2026.
- [9] H. Ye et al. *Evaluation-Driven Scaling for Scientific Discovery (SimpleTES)*. arXiv:2604.19341, 2026.
- [10] A. Ouyang, S. Guo, S. Arora, A. L. Zhang, W. Hu, C. Ré, A. Mirhoseini. *KernelBench: Can LLMs Write Efficient GPU Kernels?* arXiv:2502.10517, 2025.
- [11] B. Romera-Paredes et al. *Mathematical Discoveries from Program Search with Large Language Models*. *Nature* 625(7995):468–475, 2024.
- [12] J. Zhang, S. Hu, C. Lu, R. Lange, J. Clune. *Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents*. ICLR 2026.
- [13] G. Gerganov et al. *llama.cpp*. <https://github.com/ggml-org/llama.cpp>.
- [14] T. Zheng et al. *Dream-RSI official repository*. <https://github.com/zhengkid/Dream-RSI> (accessed 2026-09-19; README states code release pending).
- [15] J. Nam, J. Yoon, Y. Pan, Y. Wang, R. Meng, P. Ranganathan, T. Pfister. *ScientistTwo: Pioneering the Human Knowledge Frontier with Autonomous AI*. arXiv:2609.19644, 2026.

Verification note. Reference [1] was checked against its arXiv abstract page (submitted 14 September 2026) and against the PDF hosted on the project page. Reference [14] was checked directly. References [2]–[12] are given as they appear in the bibliography of [1]. Reference [13] is the upstream repository of the inference server assumed in §4.2. Reference [15] was checked against its arXiv PDF (v1, 17 September 2026).

Appendix A. Node record schema

Each line of `trees/roundRR.jsonl` is one JSON object:

Field	Type	Meaning
id, seq, round	str, int, int	rRRnSSSS, sequence, round
max_parallelism	int	Root only: W in force when the tree was recorded
parent, depth	str null, int	Parent node id; root has depth 0
task, dir	str, str	Assignment id and agent directory (null task at the root)
status	str	root, success, partial, failed_validation, error
heuristic_score, score_parts	float, dict	§8.1 and its components
score	float	Final evaluator score s_v (§8.2), fixed at creation
test_pass_rate, test_count, tests_passed	float null, int, int	Evaluator test outcome (per test file)
test_failures	list	Up to five failure summaries (file, status, message)
gain	float null	score – parent’s score, for continuations
cost, attempts	int	Agent calls charged, including retries
support	bool	Created by an off-policy support probe
files, file_hashes	list	Paths relative to <code>work/</code> ; normalized SHA-256 of contents
emitted, inherited	int	Files written by this attempt; files copied from parent
violations, notes	list	Scope/contract violations; outgoing notes (recipient, length)
log_path	str null	Relative path of the per-attempt log
truncated, finish_reason, raw_chars	bool, str, int	Output-limit diagnostics
elapsed, prompt_tokens, completion_tokens, tps, is_estimated, slot	—	Performance telemetry
score_file_level, integration_q, integration	float, float, dict	File-level score; whole-project q ; groups and details (§17.4)
integration_full_q, integration_baseline, integration_baseline_q, integration_delta, integration_credit	—	Credit computation (§17.4)
seeded_from	str null	Earlier attempt (or s00tNN seed) this carry-forward root continues
lineage_stalls, lineage_dead, lineage_verdict	int, bool, str	Line-of-work verdict (§17.8)
dependency_violations, shortcut_hits, library_misuse	list	Gate findings (§17.3)
evaluation_text	str	The evaluation appended to the attempt’s log

Appendix B. Event types (`comms/events.jsonl`)

event	Emitted when	Key fields
roster	Partition persisted	agents
node	Every attempt, after evaluation	round, node, parent, task, depth, status, cost, score, gain, files, tests
violation	Each violation of an attempt	node, agent, attempt, detail
policy_failure	Deployed policy failed online	detail
policy_missing	Policy file absent for round > 1	fallback
support	Support probes finished (extension)	reserved, spent
reconcile	Reconciliation written	nodes, dup_content, dup_names, violations, failed, truncated, files, coverage, best_mean
test_result	Each evaluator test execution	agent, node, artifact, status

event	Emitted when	Key fields
dream	Dream completed	candidates, valid, winner, winner_score, baseline_score, improved
round_reset	Interrupted round archived	archived
integration	Attempt integrated	agent, node, q, groups
credit	Credit computed	agent, node, q_with, q_base, baseline, delta
dependency_reject	Import outside the container	agent, node, modules
shortcut_penalty	Metric set from a flag or to a constant	agent, node, hits
library_misuse	Missing member or wrong arguments	agent, node, items
policy_inherited	Round-1 policy inherited	from
coverage_guard	Deployed policy skipped assignments	covered, tasks
skeptic_review	Final review done	valid, suspect, invalid, control
rebuttal	Rebuttal round started	targets, budget
final_writeup_refresh	Write-up refreshed	deliverable, node, ungrounded_numbers

Every event carries an ISO-8601 ts.

Appendix C. Citing

When reporting results obtained with this pipeline, cite the method [1] and identify this implementation (ThereminQ autoresearch-rsi-onefile.py, with commit), together with the adaptations listed in §13.2.

```
@article{zheng2026dreamrsi,
  title   = {Dream-RSI: Recursive Self-Improvement through Evolving Worlds},
  author  = {Zheng, Tong and Wu, Xidong and Zhang, Zheng and He, Zhankui and
            Zhang, Chaoyi and Coleman, Benjamin and Wei, Ruoqiao and Bai, Di and
            Liu, HaoLin and Liu, Rui and Wang, Xue and Zhuan, Yue and
            Kang, Wang-Cheng and Xiang, Renkai and Huang, Heng and
            Cheng, Xinwu and Guo, Yunsong},
  journal = {arXiv preprint arXiv:2609.14858},
  year    = {2026}
}
```